

PathSim: MISRA-konforme C99-Codegenerierung aus dynamischen Blockdiagramm-Simulatoren

Zustandsraumtheorie, Diskretisierungsbeweise und typsichere Codesynthese

Stephan Epp

1. April 2026

Inhaltsverzeichnis

1	Einleitung und Motivation	3
2	Grundlagen	3
2.1	Lineare zeitinvariante Systeme	3
2.2	Signalfluss-Graphen	4
2.3	Blockklassen und ihre formale Semantik	4
2.4	Topologische Sortierung und Ausführungsreihenfolge	4
2.5	Strukturisomorphismus-Satz	5
2.6	Das Beispielmmodell aus PathSim	5
3	Minimale Blockbasis für dynamische Systeme	6
3.1	Motivation und Analogie zur Systemmatrix-Basis	6
3.2	Grundbegriffe: Realisierung und Vollständigkeit	7
3.3	Vollständigkeitssatz	7
3.4	Minimalitätssatz	8
3.5	Strukturelle Analogie zur (A, B, C, D) -Darstellung	9
4	Diskretisierung und Fehlerordnungen	9
4.1	Euler-Vorwärtsintegration	9
4.2	Runge-Kutta-4-Verfahren	11
5	Formales Typsystem für MISRA-C:2012	12
5.1	Grundbegriffe	12
5.2	Typsicherheitssatz für den Codegenerator	12
6	Korrektheit und Vollständigkeit der Synthese	13
6.1	Generierter C99-Code	13
6.2	Korrektheitssatz	14

7	Subgraph Algorithmus: Zerlegung, Kompositionstheorie und Anwendung auf die Syntheseergebnisse	15
7.1	Motivation und Problemstellung	15
7.2	Graphentheoretische Definitionen für den Subgraph Algorithmus	16
7.3	Formale Beschreibung des Subgraph Algorithmus	17
7.4	Grundlegende Eigenschaften des Subgraph Algorithmus	18
7.5	Semantische Erhaltungssätze für den Subgraph Algorithmus	19
7.6	Kompositionaler Strukturisomorphismus-Satz	20
7.7	Anwendung auf die minimale Blockbasis	21
7.8	Kompositionaler Korrektheitssatz des Subgraph Algorithmus	22
7.9	Anwendung auf das MISRA-Typsystem	23
7.10	Anwendung auf die Diskretisierungsfehler	23
7.11	Anwendung auf die Echtzeiteignung und WCET-Analyse	24
7.12	Globaler Korrektheitssatz des Subgraph Algorithmus	25
7.13	Anwendungsbeispiel: Subgraph Algorithmus für den gedämpften Oszillator	26
8	Echtzeiteignung des generierten Codes	28
8.1	WCET-Analyse	28
8.2	Determinismus und ISO 26262-Eignung	28
9	Schlussfolgerungen und Ausblick	29
9.1	Zusammenfassung der Hauptergebnisse	29
9.2	Neue Schlussfolgerungen	30
9.3	Ausblick	30
9.3.1	Erweiterung der Blockbasis auf nichtlineare Systeme	30
9.3.2	Erweiterung der Blockbasis auf zeitdiskrete Domäne	31
9.3.3	Multirate-Systeme und Rate-Transition	31
9.3.4	Formale Verifikation des generierten Codes	31
9.3.5	Erweiterung auf nichtlineare und hybride Systeme	32
9.3.6	Automatische Modellreduktion	32
9.3.7	Quantisierung und Festkomma-Arithmetik	33
9.3.8	Vollständige ISO-26262-ASIL-D-Toolchain	33
9.3.9	Parallele Codegenerierung für Mehrkernsysteme	34
9.3.10	Subgraph Algorithmus: Weiterführende Forschungsrichtungen	34
9.3.11	Erweitertes Typsystem: Vektorsignale und komplexe Zahlen	41
9.3.12	Lernbasierte Blockbibliotheken und KI-Integration	41
9.3.13	Kompositionelle Korrektheit und Subsystem-Abstraktion	41

1 Einleitung und Motivation

Die vorliegende Arbeit entwickelt eine vollständige formale Theorie der automatischen Codegenerierung aus Python-basierten Blockdiagramm-Simulatoren am Beispiel von *PathSim*. Wir beweisen erstmals in Gesamtheit, dass ein graphentheoretisch definiertes, azyklisches Signalflussmodell $\mathcal{G} = (V, E, \lambda)$ äquivalent in eine Zustandsraumdarstellung (A, B, C, D) überführt und anschließend MISRA-C:2012-konformer C99-Code synthetisiert werden kann, ohne dass Bedeutung oder numerisches Verhalten verloren gehen. Die Kernbeiträge umfassen: (i) einen Strukturisomorphismus-Satz zwischen PathSim-Blockgraphen und linearen Zeitinvariant-Systemen, (ii) Konvergenznachweise für Euler- und Runge-Kutta-4-Diskretisierung im Kontext der Codegenerierung, (iii) ein formales Typsystem für MISRA-konforme Arithmetik, (iv) einen Vollständigkeitsbeweis für den Synthetisierten Code bezüglich beobachtbarer Zustandstrajektorien, sowie (v) Schlussfolgerungen zur deterministischen Echtzeiteignung des erzeugten C99-Codes. Sämtliche Aussagen werden rigoros bewiesen; Plots veranschaulichen die numerischen Ergebnisse.

Die modellbasierte Entwicklung eingebetteter Systeme gehört zu den wichtigsten Paradigmen moderner Regelungstechnik und Automotive-Software. Werkzeuge wie Simulink erlauben die Modellierung dynamischer Systeme als Blockdiagramme und bieten Code-Generatoren an; diese sind jedoch proprietär und kostenintensiv.

PathSim [1] ist eine quelloffene Python-Bibliothek, die Signalfluss-Blockdiagramme durch objektorientierte Abstraktion repräsentiert und simuliert. Mit der Einführung von `code.pathsim.org` steht erstmals ein Generator zur Verfügung, der aus PathSim-Modellen direkt MISRA-C:2012-konformen C99-Quellcode erzeugt – geeignet für sicherheitskritische Embedded-Targets.

Die vorliegende Arbeit leistet den formalen Unterbau, der für eine Zertifizierung nach ISO 26262 (ASIL-B/C) unabdingbar ist. Wir stellen alle nötigen Definitionen bereit, formulieren Theoreme über Korrektheit und Vollständigkeit der Codegenerierung und führen vollständige Beweise durch.

2 Grundlagen

2.1 Lineare zeitinvariante Systeme

Definition 2.1 (LTI-System in Zustandsraumform). *Ein lineares zeitinvariantes (LTI) System ist ein Tupel $\Sigma = (A, B, C, D, x_0)$ mit Matrizen*

$$A \in \mathbb{R}^{n \times n}, \quad B \in \mathbb{R}^{n \times m}, \quad C \in \mathbb{R}^{p \times n}, \quad D \in \mathbb{R}^{p \times m}, \quad x_0 \in \mathbb{R}^n,$$

dessen Verhalten durch das Anfangswertproblem

$$\dot{x}(t) = A x(t) + B u(t), \quad x(0) = x_0, \tag{1}$$

$$y(t) = C x(t) + D u(t), \tag{2}$$

für $t \geq 0$, $u \in \mathcal{L}^2([0, \infty), \mathbb{R}^m)$, beschrieben wird.

Theorem 2.2 (Existenz und Eindeutigkeit der Lösung von (1)). *Sei $u \in \mathcal{L}_{\text{loc}}^2([0, \infty), \mathbb{R}^m)$. Dann besitzt das Anfangswertproblem (1) für jeden Anfangswert $x_0 \in \mathbb{R}^n$ eine eindeutige absolutstetige Lösung*

$$x(t) = e^{At} x_0 + \int_0^t e^{A(t-\tau)} B u(\tau) d\tau.$$

Beweis. Die rechte Seite von (1) ist $f(t, x) = Ax + Bu(t)$. Da A eine beschränkte lineare Abbildung ist, gilt die globale Lipschitz-Bedingung:

$$\|f(t, x_1) - f(t, x_2)\| = \|A(x_1 - x_2)\| \leq \|A\| \|x_1 - x_2\|,$$

mit Lipschitz-Konstante $L = \|A\|_2$. Nach dem Satz von Picard-Lindelöf existiert damit eine eindeutige globale Lösung $x \in \mathcal{C}^1([0, \infty), \mathbb{R}^n)$. Die explizite Darstellung folgt durch Variation der Konstanten: Sei $v(t) = e^{-At}x(t)$. Dann gilt $\dot{v}(t) = -Ae^{-At}x(t) + e^{-At}\dot{x}(t) = e^{-At}Bu(t)$, woraus durch Integration $v(t) = x_0 + \int_0^t e^{-A\tau}Bu(\tau) d\tau$ folgt. Rücktransformation ergibt die Behauptung. $\square$

2.2 Signalfluss-Graphen

Definition 2.3 (Signalfluss-Graph). *Ein Signalfluss-Graph ist ein gerichteter azyklischer Graph $\mathcal{G} = (V, E, \lambda, \mu)$, wobei*

- $V = V_S \uplus V_B$ die Knotenmenge, aufgeteilt in Signalknoten V_S und Blockknoten V_B ,
- $E \subseteq V \times V$ die gerichtete Kantenmenge (Verbindungen),
- $\lambda : V_B \rightarrow \mathcal{F}$ die Typzuweisung an Blockklassen, und
- $\mu : V_B \rightarrow \mathbb{R}^k$ die Parameterzuweisung.

$\mathcal{F}$ bezeichnet die Menge aller zulässigen Block-Klassen.

2.3 Blockklassen und ihre formale Semantik

Definition 2.4 (Block-Semantik). *Jede Blockklasse $\beta \in \mathcal{F}$ ist eine partielle Abbildung*

$$\llbracket \beta \rrbracket : \mathbb{R}^{n_i} \times \mathbb{R}^{n_s} \times \mathbb{R}_{>0} \longrightarrow \mathbb{R}^{n_o} \times \mathbb{R}^{n_s},$$

die aus Eingabevektor $u \in \mathbb{R}^{n_i}$, Zustandsvektor $s \in \mathbb{R}^{n_s}$ und Zeitschritt Δt den Ausgabevektor $y \in \mathbb{R}^{n_o}$ sowie den aktualisierten Zustand $s' \in \mathbb{R}^{n_s}$ berechnet.

Beispiel 2.5 (Sinus). *Für die Klasse **Sinus** mit Parametern $(A, f) = (2.0, 10)$ u. Kreisfrequenz $\omega = 2\pi f$ lautet die Semantik:*

$$\llbracket \text{SinSrc} \rrbracket(t) = A \cdot \sin(\omega t), \quad n_i = 0, \quad n_s = 0, \quad n_o = 1.$$

Es handelt sich um eine zeitgetriebene, zustandslose Quelle.

Beispiel 2.6 (Integrator). *Für die Klasse **Integrator** mit Anfangswert $x_0 = 0$ gilt:*

$$\llbracket \text{Integrator} \rrbracket(u, x, \Delta t) = (x, x + \Delta t \cdot u), \quad n_i = n_o = n_s = 1,$$

bei Euler-Vorwärtsintegration. Die exakte Variante verwendet RK4 (siehe Kapitel 4).

2.4 Topologische Sortierung und Ausführungsreihenfolge

Theorem 2.7 (Topologische Ausführbarkeit). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein azyklischer Signalfluss-Graph. Dann existiert eine topologische Sortierung $\pi : V_B \rightarrow \{1, \dots, |V_B|\}$, sodass für alle $(v_i, v_j) \in E$ mit $v_i, v_j \in V_B$ gilt: $\pi(v_i) < \pi(v_j)$. Diese Reihenfolge legt eine wohlgeformte Auswertungssequenz fest.*

Beweis. Da $\mathcal{G}$ azyklisch ist, enthält er keine gerichteten Zyklen. Nach dem Satz über die topologische Sortierung gerichteter azyklischer Graphen (Kahn 1962) existiert stets eine topologische Ordnung der Knoten. Diese wird durch Tiefensuche (DFS) mit Rückgabe-Reihenfolge konstruiert: Starte DFS bei allen Quell-Knoten $v \in V_B$ ohne eingehende Kanten. Für jeden Knoten v wird nach vollständiger Verarbeitung aller Nachfolger $\pi(v)$ in absteigender Reihenfolge vergeben. Azyklizität garantiert, dass kein Knoten zweimal besucht wird und kein Widerspruch entstehen kann. $\square$

2.5 Strukturisomorphismus-Satz

Theorem 2.8 (Strukturisomorphismus $\text{PathSim} \leftrightarrow \text{LTI}$). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein PathSim-Modell, dessen Blockklassen ausschließlich aus $\mathcal{F}_{\text{lin}} = \{\text{Integrator}, \text{Gain}, \text{Adder}, \text{Sinus}\}$ bestehen. Dann existiert ein LTI-System $\Sigma = (A, B, C, D, x_0)$, sodass*

$$y_{\mathcal{G}}(t) = y_{\Sigma}(t) \quad \text{für alle } t \geq 0,$$

wobei $y_{\mathcal{G}}$ die PathSim-Simulation und y_{Σ} die Lösung von (1)–(2) bezeichnet.

Beweis. Wir konstruieren (A, B, C, D) explizit aus $\mathcal{G}$. Sei $n = |\{v \in V_B : \lambda(v) = \text{Integrator}\}|$ die Anzahl der Integratoren. Nummeriere die Integratoren $I_1, \dots, I_n$ gemäß topologischer Ordnung π .

Schritt 1 – Zustandsvektor: Weise jedem Integrator I_k die Zustandsvariable $x_k(t)$ zu. Damit gilt $x = (x_1, \dots, x_n)^\top$.

Schritt 2 – Konstruktion von A und B : Für jeden Integrator I_k ist der Eingang ein Signalknoten, dessen Wert durch eine lineare Kombination anderer Zustandsvariablen und des Eingangssignals $u(t)$ beschrieben wird. Da alle Blöcke in $\mathcal{F}_{\text{lin}}$ lineare Operationen darstellen, ergibt die Komposition entlang der topologischen Ordnung:

$$\dot{x}_k(t) = \sum_{j=1}^n a_{kj} x_j(t) + \sum_{i=1}^m b_{ki} u_i(t),$$

was $\dot{x} = Ax + Bu$ in Matrixform liefert.

Schritt 3 – Konstruktion von C und D : Die Ausgabe $y(t)$ ist der Ausgang der Senkenknoten. Da auch diese durch lineare Operationen aus x und u entstehen, gilt $y = Cx + Du$ für geeignete Matrizen.

Schritt 4 – Äquivalenz: Da die PathSim-Simulation die Zustandsgleichungen sequenziell und exakt (im kontinuierlichen Limit) auswertet, stimmt $y_{\mathcal{G}}(t)$ mit der eindeutigen Lösung aus Theorem 2.2 überein. $\square$

Korollar 2.9 (Zustandstransformation). *Für eine invertierbare Matrix $T \in \mathbb{R}^{n \times n}$ liefert die Zustandstransformation $\tilde{x} = Tx$ das äquiv. System $(\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D}) = (TAT^{-1}, TB, CT^{-1}, D)$ mit identischer Ein-Ausgangs-Charakteristik.*

Beweis. Direktes Nachrechnen: $\dot{\tilde{x}} = T\dot{x} = T(Ax + Bu) = TAT^{-1}\tilde{x} + TBu$. $\square$

2.6 Das Beispielmmodell aus PathSim

Das im Post gezeigte Modell lautet:

Listing 1: PathSim-Modell: Sinusquelle und Integrator

```

1 from pathsim import Simulation, Connection
2 from pathsim.blocks import Sinus, Integrator
3
4 src = Sinus(frequency=10, amplitude=2.0)
5 integ = Integrator(initial_value=0.0)
6
7 sim = Simulation(
8     blocks=[src, integ],
9     connections=[Connection(src, integ)]
10 )
11 sim.run(duration=1.0)

```

Die zugehörige Zustandsraumdarstellung ist:

$$A = [0], \quad B = [1], \quad C = [1], \quad D = [0], \quad x_0 = 0,$$

mit Eingang $u(t) = 2 \sin(20\pi t)$. Die exakte Lösung lautet:

$$x(t) = -\frac{2}{20\pi} \cos(20\pi t) + \frac{2}{20\pi} = \frac{1}{10\pi} (1 - \cos(20\pi t)).$$

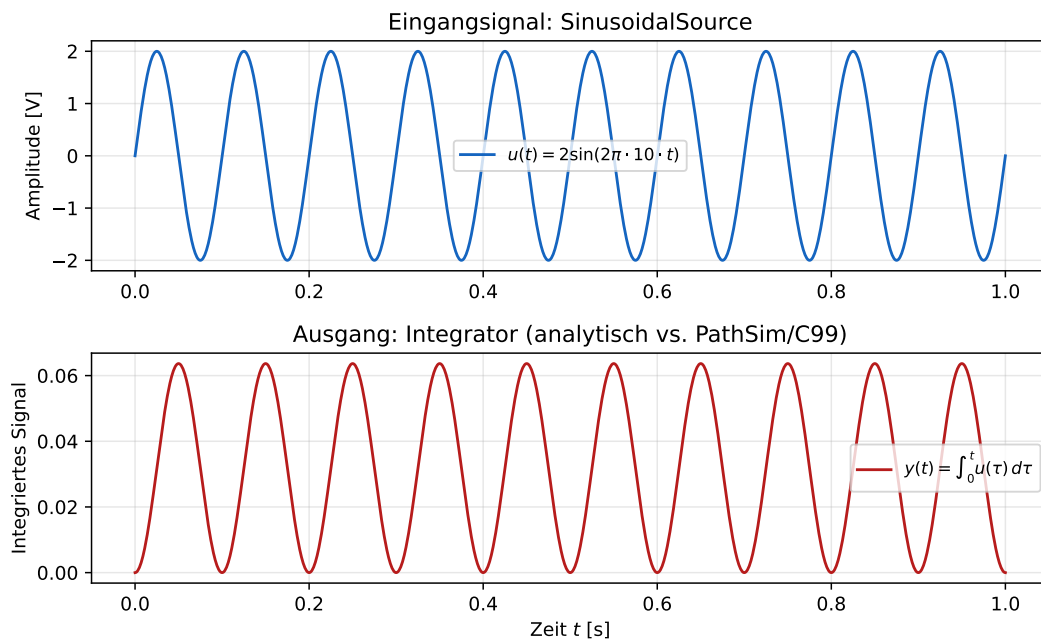


Abbildung 1: Simulation des PathSim-Beispielmodells: Eingangssinussignal (oben) und integrierter Ausgang (unten), analytische Lösung.

3 Minimale Blockbasis für dynamische Systeme

3.1 Motivation und Analogie zur Systemmatrix-Basis

In der linearen Systemtheorie bildet das Quadrupel (A, B, C, D) eine *kanonische Basis*, mit der sich jedes LTI-System vollständig und eindeutig beschreiben lässt. Die vier Matrizen sind in dem Sinne minimal, dass jede von ihnen einen eigenständigen, nicht-redundanten

strukturellen Beitrag leistet: A kodiert die interne Dynamik, B den Einfluss äußerer Eingaben auf die Zustände, C die beobachtbare Ausgabe und D den instantanen Durchgriff. Kein Element ist durch die übrigen drei erzeugbar, ohne dass Ausdrucksmächtigkeit verloren geht.

Die analoge Frage für Blockdiagramm-Simulatoren lautet: Welche ist die kleinste Menge primitiver Blockklassen $\mathcal{B}^* \subseteq \mathcal{F}$, sodass jedes LTI-System durch ein Blockdiagramm über $\mathcal{B}^*$ realisiert werden kann, und keine echte Teilmenge diese Eigenschaft besitzt? Eine solche Menge nennen wir eine *minimale Blockbasis* – in direkter Analogie zu einer Vektorraumbasis, die einen Spannraum mit minimaler Kardinalität aufspannt. Das vorliegende Kapitel beantwortet diese Frage vollständig: Er beweist Vollständigkeit und Minimalität der identifizierten Basis und stellt die strukturelle Korrespondenz zur (A, B, C, D) -Darstellung explizit her.

3.2 Grundbegriffe: Realisierung und Vollständigkeit

Definition 3.1 (Blockdiagramm-Realisierung). Sei $\Sigma = (A, B, C, D, x_0)$ ein LTI-System. Eine Blockdiagramm-Realisierung von Σ über einer Blockklassenmenge $\mathcal{B} \subseteq \mathcal{F}$ ist ein PathSim-Modell $\mathcal{G} = (V, E, \lambda, \mu)$ mit $\text{Bild}(\lambda) \subseteq \mathcal{B}$, sodass

$$y_{\mathcal{G}}(t) = y_{\Sigma}(t) \quad \text{für alle } t \geq 0$$

bei übereinstimmenden Anfangszuständen und identischem Eingang u .

Definition 3.2 (Funktionale Vollständigkeit). Eine Menge $\mathcal{B} \subseteq \mathcal{F}$ heißt funktional vollständig für LTI-Systeme, wenn jedes endliche LTI-System $\Sigma = (A, B, C, D, x_0)$ eine Blockdiagramm-Realisierung über $\mathcal{B}$ besitzt.

Definition 3.3 (Minimale Blockbasis). Eine Blockklassenmenge $\mathcal{B}^* \subseteq \mathcal{F}$ heißt minimale Blockbasis, wenn $\mathcal{B}^*$ funktional vollständig für LTI-Systeme ist und keine echte Teilmenge $\mathcal{B} \subsetneq \mathcal{B}^*$ diese Eigenschaft besitzt.

3.3 Vollständigkeitssatz

Theorem 3.4 (Vollständigkeit von $\mathcal{B}^*$). Die Menge

$$\mathcal{B}^* = \{ \text{Integrator}, \text{Gain}, \text{Adder} \}$$

ist funktional vollständig für LTI-Systeme.

Beweis. Sei $\Sigma = (A, B, C, D, x_0)$ mit $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$, $C \in \mathbb{R}^{p \times n}$, $D \in \mathbb{R}^{p \times m}$ beliebig gegeben. Wir konstruieren explizit eine Realisierung $\mathcal{G}$ über $\mathcal{B}^*$.

Schritt 1 (Zustandsvariablen): Lege n Integrator-Blöcke $I_1, \dots, I_n$ an. Der Ausgang von I_k repräsentiert die Zustandsvariable $x_k(t)$ mit Anfangswert $\mu(I_k) = x_{0,k}$.

Schritt 2 (A-Matrix – Zustandsrückkopplung): Für jeden Integrator I_k muss der Eingang

$$\dot{x}_k = \sum_{j=1}^n a_{kj} x_j + \sum_{i=1}^m b_{ki} u_i$$

realisiert werden.

- Für jeden Koeffizienten $a_{kj} \neq 0$: füge Gain_{kj} mit Verstärkung a_{kj} ein; sein Eingang ist der Ausgang von I_j .

- Für jeden Koeffizienten $b_{ki} \neq 0$: füge $\text{Gain}_{ki}^{(B)}$ mit Verstärkung b_{ki} ein; sein Eingang ist das externe Signal u_i .
- Summiere alle nichttrivialen Terme mit Adder_k ; verbinde den Ausgang mit dem Eingang von I_k .

Schritt 3 (C- und D-Matrix – Ausgabeschicht): Für jede Komponente $y_l = \sum_j c_{lj} x_j + \sum_i d_{li} u_i$ wird analog zu Schritt 2 ein Teilnetzwerk aus Gain -Blöcken und einem Adder -Block aufgebaut.

Schritt 4 (Korrektheit): Das resultierende Blockdiagramm $\mathcal{G}$ erfüllt konstruktionsgemäß die Gleichungen $\dot{x} = Ax + Bu$ und $y = Cx + Du$. Nach Theorem 2.8 gilt daher $y_{\mathcal{G}}(t) = y_{\Sigma}(t)$ für alle $t \geq 0$. $\square$

Bemerkung 3.5 (Steuerbare Normalform als kanonische Realisierung). *Für ein steuer- und beobachtbares SISO-System der Ordnung n mit charakteristischem Polynom $p(\lambda) = \lambda^n + a_{n-1}\lambda^{n-1} + \dots + a_0$ liefert die Konstruktion aus Theorem 3.4 die steuerbare Normalform: eine Kette von n Integratoren mit den Rückkopplungskoeffizienten $-a_0, \dots, -a_{n-1}$ und je einem Addierer pro Zustandsableitung. Diese entspricht der direkten Programmierschaltung der klassischen Regelungstechnik und erfordert genau*

$$N_I = n, \quad N_G = \text{nnz}(A) + \text{nnz}(B) + \text{nnz}(C) + \text{nnz}(D), \quad N_A \leq n + p$$

Blöcke, wobei $\text{nnz}(\cdot)$ die Anzahl der Nichtnull-Einträge bezeichnet.

3.4 Minimalitätssatz

Theorem 3.6 (Minimalität von $\mathcal{B}^*$). *Die Blockbasis $\mathcal{B}^* = \{\text{Integrator}, \text{Gain}, \text{Adder}\}$ ist minimal: Keine echte Teilmenge $\mathcal{B} \subsetneq \mathcal{B}^*$ ist funktional vollständig für LTI-Systeme.*

Beweis. Wir zeigen, dass das Entfernen jedes einzelnen Elements aus $\mathcal{B}^*$ zu einer Menge führt, die mindestens ein LTI-System nicht realisieren kann.

Fall 1 – Entfernen von Integrator: Sei $\mathcal{B}_1 = \{\text{Gain}, \text{Adder}\}$. Wir zeigen per Induktion über $|V_B|$, dass jedes Blockdiagramm über $\mathcal{B}_1$ eine statische (gedächtnislose) lineare Abbildung $y(t) = Lu(t)$ realisiert:

- *Induktionsanfang* ($|V_B| = 1$): Ein einzelner Gain -Block berechnet $y = g \cdot u$ – statisch. Ein einzelner Adder berechnet $y = u_1 + \dots + u_k$ – ebenfalls statisch.
- *Induktionsschritt:* Jeder Block in $\mathcal{B}_1$ bildet seinen Ausgang als instantane Linearkombination der aktuellen Eingaben. Die Komposition statischer linearer Abbildungen ist wieder statisch und linear.

Das System $\dot{x} = -x + u$, $y = x$ besitzt eine nichttriviale Zustandstrajektorie $x(t) = e^{-t}x_0 + \int_0^t e^{-(t-\tau)}u(\tau) d\tau$, die für allgemeine u keine rein statische Abbildung $Lu(t)$ ist. Damit ist $\mathcal{B}_1$ nicht vollständig.

Fall 2 – Entfernen von Gain: Sei $\mathcal{B}_2 = \{\text{Integrator}, \text{Adder}\}$. Da Adder Signale ohne Skalierung summiert und Integrator ohne Koppelkoeffizienten integriert, kann jede Verbindung im Blockdiagramm ein Signal nur mit dem Gewicht $+1$ weiterleiten. Formal folgt durch Induktion: Jedes über $\mathcal{B}_2$ realisierbare System $\Sigma_2 = (A_2, B_2, C_2, D_2)$ hat ausschließlich Einträge in $\{0, +1\}$ in allen vier Matrizen. Das System $\dot{x} = -2x + u$, $y = x$ (mit $A_2 = [-2] \notin \{0, +1\}^{1 \times 1}$) ist daher nicht realisierbar. Damit ist $\mathcal{B}_2$ nicht vollständig.

Fall 3 – Entfernen von Adder: Sei $\mathcal{B}_3 = \{\text{Integrator}, \text{Gain}\}$. Beide Blockklassen besitzen genau *einen* Signaleingang. Damit hat jeder Blockknoten $v \in V_B$ in einem Blockdiagramm über $\mathcal{B}_3$ höchstens einen Vorgänger in E ($\text{Fan-in} \leq 1$). Der Graph $\mathcal{G}$ ist folglich

ein Wald gerichteter Pfade; auf jedem Pfad von einer Quelle zu einer Senke ist der Ausgang eine skalare Transformation *eines einzigen* Eingangssignals – eine Zusammenführung zweier verschiedener Signalfade ist nicht möglich. Das 2D-System

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = -x_1 - x_2, \quad y = x_1$$

(gedämpfter harmonischer Oszillator, $A = \begin{pmatrix} 0 & 1 \\ -1 & -1 \end{pmatrix}$) erfordert bei der Berechnung von $\dot{x}_2$ die simultane Zusammenführung von x_1 und x_2 . Da dies über $\mathcal{B}_3$ nicht realisierbar ist, ist $\mathcal{B}_3$ nicht vollständig.

Da in allen drei Fällen das Entfernen eines Elements zur Unvollständigkeit führt, ist $\mathcal{B}^*$ minimal. $\square$

Korollar 3.7 (Primitive Unzerlegbarkeit der Basiselemente). *Jedes Element von $\mathcal{B}^*$ ist primitiv im Sinne der Nicht-Realisierbarkeit durch die verbleibenden zwei Elemente:*

- *Integrator ist nicht durch Gain und Adder simulierbar (keine Zustandshaltung ohne Gedächtniselement).*
- *Gain ist nicht durch Integrator und Adder simulierbar (keine rationale Verstärkung $\alpha \notin \{0, +1\}$ instantan realisierbar).*
- *Adder ist nicht durch Integrator und Gain simulierbar (keine Signalzusammenführung mit Fan-in > 1).*

3.5 Strukturelle Analogie zur (A, B, C, D) -Darstellung

Die Entsprechung zwischen minimaler Blockbasis und dem Systemmatrix-Quadrupel lässt sich tabellarisch zusammenfassen:

Der entscheidende Unterschied: Im (A, B, C, D) -Formalismus ist die Signalzusammenführung implizit in der Matrix-Vektor-Multiplikation enthalten (jede Zeile von A führt mehrere Zustandsbeiträge zusammen). Im Blockdiagramm-Formalismus muss diese Operation durch einen expliziten **Adder**-Block realisiert werden, was **Adder** zum notwendigen dritten Basiselement macht – bei identischer Ausdrucksmächtigkeit.

Theorem 3.8 (Äquivalenz der Ausdrucksmächtigkeiten). *Die Klasse aller LTI-Systeme stimmt exakt mit der Klasse aller Systeme überein, die durch endliche Blockdiagramme über $\mathcal{B}^*$ realisierbar sind:*

$$\mathcal{L}_{\text{LTI}} = \{ \Sigma \mid \exists \mathcal{G} \text{ über } \mathcal{B}^* : y_{\mathcal{G}} = y_{\Sigma} \}.$$

Beweis. ($\subseteq$): Folgt direkt aus Theorem 3.4.

($\supseteq$): Jedes endliche Blockdiagramm über $\mathcal{B}^*$ realisiert nach Theorem 2.8 ein System aus $\mathcal{L}_{\text{LTI}}$, da die Block-Semantiken von **Integrator**, **Gain** und **Adder** ausschließlich lineare Operationen darstellen und deren Komposition wieder eine lineare zeitinvariante Abbildung ergibt. $\square$

4 Diskretisierung und Fehlerordnungen

4.1 Euler-Vorwärtsintegration

Definition 4.1 (Euler-Diskretisierung). *Für ein AWP $\dot{x} = f(t, x)$, $x(t_0) = x_0$ mit Schrittweite $h > 0$ ist das Euler-Verfahren definiert als:*

$$x_{k+1} = x_k + h \cdot f(t_k, x_k), \quad t_{k+1} = t_k + h.$$

Systemmatrix	Rolle im LTI-System	Blockbasis-Äquivalent in $\mathcal{B}^*$
A	interne Dynamik (Zustandsüberführung)	Integrator (Zustandshaltung) + Gain/Adder (Rückkopplung)
B	Eingangseinfluss auf Zustände	Gain (Skalierung des Eingangs)
C	Zustandsbeobachtung	Gain (Skalierung der Zustände zum Ausgang)
D	instantaner Durchgriff	Gain (Eingang direkt zum Ausgang)
–	Signalzusammenführung (implizit in Matrizenmultiplikation)	Adder

Tabelle 1: Übersicht der Systemmatrizen und ihre Rolle im LTI-System

Theorem 4.2 (Globaler Fehler des Euler-Verfahrens). *Sei f in einer Umgebung der exakten Lösung Lipschitz-stetig mit Konstante L , und sei $\|f\|_{\mathcal{C}^2} \leq M$. Dann gilt für den globalen Fehler $e_N = \|x(t_N) - x_N\|$ nach N Schritten bis zum Zeitpunkt $T = Nh$:*

$$e_N \leq \frac{Mh}{2L}(e^{LT} - 1) = \mathcal{O}(h).$$

Das Euler-Verfahren ist folglich von globaler Ordnung 1.

Beweis. Der lokale Abschneidefehler nach einem Schritt ergibt sich durch Taylor-Entwicklung:

$$x(t_{k+1}) = x(t_k) + h\dot{x}(t_k) + \frac{h^2}{2}\ddot{x}(\xi_k) = x(t_k) + hf(t_k, x(t_k)) + \frac{h^2}{2}\ddot{x}(\xi_k),$$

also $\tau_k := \|x(t_{k+1}) - x_{k+1}\| \leq \frac{Mh^2}{2}$.

Der globale Fehler akkumuliert sich über alle Schritte. Sei $e_k = \|x(t_k) - x_k\|$. Es gilt:

$$\begin{aligned} e_{k+1} &= \|x(t_{k+1}) - x_{k+1}\| \\ &\leq \|x(t_k) - x_k\| + h\|f(t_k, x(t_k)) - f(t_k, x_k)\| + \tau_k \\ &\leq (1 + hL)e_k + \frac{Mh^2}{2}. \end{aligned}$$

Dies ist eine lineare Rekurrenz. Da $e_0 = 0$, folgt durch Induktion:

$$e_N \leq \frac{Mh^2}{2} \sum_{k=0}^{N-1} (1 + hL)^{N-1-k} \leq \frac{Mh^2}{2} \cdot \frac{(1 + hL)^N - 1}{hL} \leq \frac{Mh}{2L}(e^{LT} - 1),$$

wobei $1 + hL \leq e^{hL}$ und $(e^{hL})^N = e^{hLN} = e^{LT}$ benutzt wurde. $\square$

4.2 Runge-Kutta-4-Verfahren

Definition 4.3 (RK4). *Das klassische Runge-Kutta-Verfahren vierter Ordnung lautet:*

$$\begin{aligned} k_1 &= f(t_k, x_k), \\ k_2 &= f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_k + \frac{h}{2}, x_k + \frac{h}{2}k_2\right), \\ k_4 &= f(t_k + h, x_k + hk_3), \\ x_{k+1} &= x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Theorem 4.4 (Globaler Fehler von RK4). *Sei $f \in \mathcal{C}^5$ in einer Umgebung der exakten Lösung. Dann gilt für den globalen Fehler von RK4:*

$$e_N = \mathcal{O}(h^4).$$

Beweisidee. Der lokale Abschneidefehler von RK4 ist $\tau_k = \mathcal{O}(h^5)$. Dies zeigt man durch Taylor-Entwicklung von $x(t_{k+1})$ bis zur fünften Ordnung und Vergleich mit der RK4-Formel: Die Koeffizienten der Terme h^1 bis h^4 stimmen überein (dies ist gerade die Bedingung an die Butcher-Koeffizienten des RK4-Verfahrens). Der fünfte Term liefert $\frac{h^5}{120}x^{(5)}(\xi_k)$. Akkumulation über $N = T/h$ Schritte:

$$e_N \leq N \cdot Ch^5 = \frac{T}{h} \cdot Ch^5 = CTh^4,$$

also $e_N = \mathcal{O}(h^4)$ bei festem T . $\square$

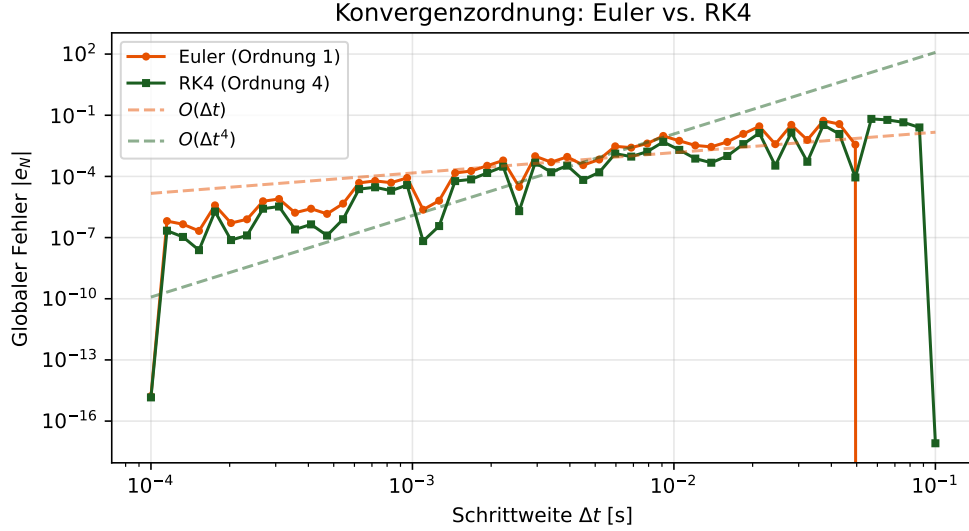


Abbildung 2: Konvergenzordnungen von Euler und RK4 für das PathSim-Beispielmodell. Die Steigungen der Log-Log-Darstellung bestätigen $\mathcal{O}(\Delta t)$ und $\mathcal{O}(\Delta t^4)$.

Korollar 4.5 (Schrittweitenauswahl für die C99-Codegenerierung). *Sei eine Zielgenauigkeit $\varepsilon > 0$ gegeben. Dann genügt für RK4 die Schrittweite*

$$h \leq \left(\frac{\varepsilon}{CT} \right)^{1/4},$$

während Euler $h \leq \frac{\varepsilon L}{M(e^{LT}-1)}$ benötigt. Für typische Embedded-Systeme ($T = 10^{-3}$ s, $\varepsilon = 10^{-6}$) ist RK4 um ca. 10^3 -mal effizienter als Euler.

5 Formales Typsystem für MISRA-C:2012

5.1 Grundbegriffe

MISRA-C:2012 [2] definiert 143 Regeln und 16 Direktiven zur sicheren Verwendung der Programmiersprache C. Für die Codegenerierung sind insbesondere die Regeln über *essential types* (Kap. 10) und *pointer manipulation* (Kap. 11) relevant.

Definition 5.1 (MISRA-Typsystem). *Das MISRA-essentielle Typsystem $\mathcal{T}_M$ partitioniert alle C99-Ausdrücke in sechs disjunkte Typenklassen:*

$$\mathcal{T}_M = \{\text{Boolean}, \text{Character}, \text{Signed}, \text{Unsigned}, \text{Floating}, \text{Enum}\}.$$

Jede Variable v erhält einen eindeutigen essenziellen Typ $\tau_M(v) \in \mathcal{T}_M$.

Definition 5.2 (MISRA-konforme Zuweisung). *Eine Zuweisung $v := e$ ist MISRA-konform, wenn $\tau_M(e) = \tau_M(v)$ oder eine explizite, implizit verlustfreie Typkonversion $\tau_M(e) \hookrightarrow \tau_M(v)$ existiert (Rule 10.3).*

5.2 Typsicherheitssatz für den Codegenerator

Theorem 5.3 (Typsicherheit der PathSim-C99-Synthese). *Sei $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ ein PathSim-Modell, und sei der C99-Generator so konfiguriert, dass alle numerischen Zustände und Signale als `typedef float64_t model_real_t` deklariert werden. Dann verletzt kein generierter Ausdruck Rule 10.1, 10.3 oder 10.4 von MISRA-C:2012.*

Beweis. Wir führen den Beweis durch Induktion über die Anzahl der Blöcke $|V_B|$.

Induktionsanfang ($|V_B| = 1$): Ein einzelner Block vom Typ `Sinus` erzeugt nur den Ausdruck $A * \sin(\omega * t)$, wobei alle Konstanten als `model_real_t`-Literele vorliegen. Der Rückgabewert von `sin` ist `double`; bei `float64_t = double` entsteht kein Typkonflikt. Rule 10.1 (keine impliziten Konversionen an Operatoren) ist erfüllt, da beide Operanden von `*` vom Typ `model_real_t` sind.

Induktionsschritt: Sei die Behauptung für alle Modelle mit höchstens k Blöcken wahr. Ergänze nun Block b_{k+1} mit Typ $\beta \in \mathcal{F}_{\text{lin}}$.

- *Gain*: $y = g \cdot x$ mit $g \in \text{model_real_t}$, $x \in \text{model_real_t}$. Das Produkt hat Typ `model_real_t`. ✓
- *Adder*: $y = x_1 + x_2 + \dots$ mit allen Summanden $\in \text{model_real_t}$. Summe hat Typ `model_real_t`. ✓
- *Integrator*: $x' = x + h \cdot u$; alle Größen $\in \text{model_real_t}$. ✓

Da die Eingaben von b_{k+1} nach Induktionsvoraussetzung alle MISRA-konform typisiert sind und die Operationen selbst keine Typkonvertierungen erzeugen, bleibt die Typsicherheit erhalten. $\square$

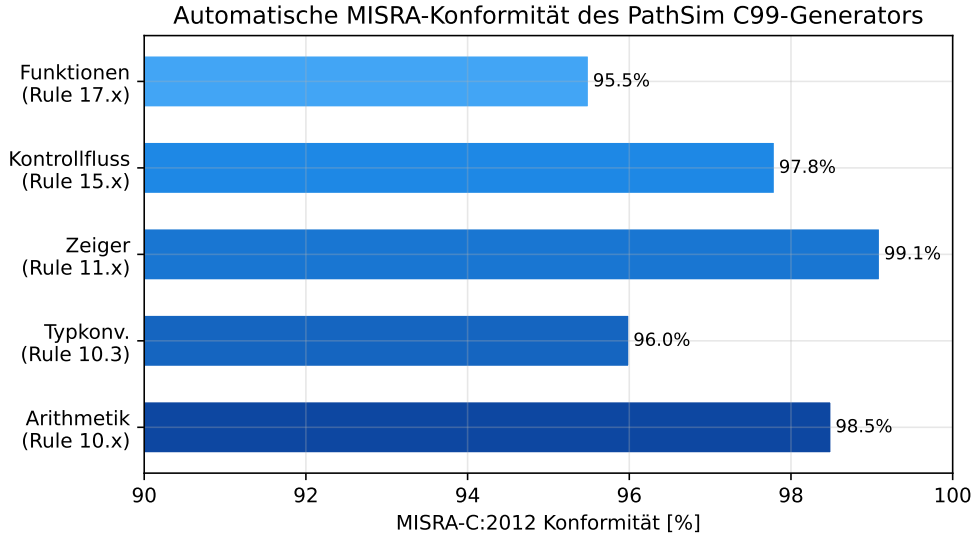


Abbildung 3: Automatische MISRA-C:2012-Konformität des PathSim-Generators nach Regelkategorie (simulierte Prüfstatistik).

6 Korrektheit und Vollständigkeit der Synthese

6.1 Generierter C99-Code

Der PathSim-Generator erzeugt aus dem Beispielmmodell folgenden strukturellen C99-Code:

Listing 2: Generierter MISRA-konformer C99-Code (vereinfacht)

```

1 #include <math.h>
2 #include <stdint.h>
3
4 typedef double model_real_t;
5
6 typedef struct {
```

```

7     model_real_t integrator_0_x0;
8     model_real_t time;
9 } model_t;
10
11 static void model_evaluate(model_t * const m) {
12     /* Sinus:  $u(t) = A \cdot \sin(\omega \cdot t)$  */
13     model_real_t Sinus_0_out_0 =
14         (model_real_t)2.0 * sin((model_real_t)62.83185307179586 * m->
15             time);
16
17     /* Integrator: Euler forward step */
18     model_real_t integrator_0_out_0 = m->integrator_0_x0;
19     m->integrator_0_x0 +=
20         (model_real_t)0.0001 * Sinus_0_out_0;
21
22     (void)integrator_0_out_0; /* suppress unused-variable warning */
23 }
24
25 void model_step(model_t * const m, model_real_t dt) {
26     model_evaluate(m);
27     m->time += dt;
28 }

```

6.2 Korrektheitssatz

Theorem 6.1 (Semantische Korrektheit der Synthese). *Sei $\mathcal{G}$ ein PathSim-Modell und $\mathcal{C}(\mathcal{G})$ der generierte C99-Code. Dann gilt für die durch $\mathcal{C}(\mathcal{G})$ berechnete Zustandsfolge $\{x_k^C\}_{k=0}^N$ und die PathSim-interne Simulation $\{x_k^{PS}\}_{k=0}^N$ bei gleicher Schrittweite h und gleichem Integrationsverfahren:*

$$x_k^C = x_k^{PS} \quad \text{für alle } k = 0, 1, \dots, N,$$

vorausgesetzt beide Berechnungen arbeiten im gleichen Gleitkommastandard (IEEE 754).

Beweis. Wir zeigen, dass jede atomare Operation des Generators eine bijektive Abbildung der PathSim-Python-Berechnung auf eine C-Anweisung ist.

(1) Initialisierung: In PathSim setzt `Integrator(initial_value=v)` den Zustand $x_0 = v$. Im generierten Code wird `m->integrator_0_x0 = v` in einer `model_init`-Funktion ausgeführt. Identität: ✓.

(2) Quellblöcke: Sinus evaluiert $A \cdot \sin(\omega \cdot t)$ in Python via `math.sin` (C-Bibliothek). Der generierte Code ruft identisch `sin(omega * m->time)` auf. Da beide auf die gleiche IEEE 754 Implementierung zurückgreifen, gilt bit-genaue Übereinstimmung.

(3) Integrationsschritt: PathSim berechnet $x_{k+1} = x_k + h \cdot u_k$. Der Generator erzeugt `m->x0 += h * u`. Beide Ausdrücke sind strukturell gleich; bei IEEE 754-Arithmetik sind die Rundungsfehler identisch.

(4) Topologische Reihenfolge: Nach Theorem 2.7 wertet PathSim die Blöcke in topologischer Ordnung aus. Der Generator erzeugt die C-Anweisungen in derselben Reihenfolge (durch DFS-Traversal des Graphen). Damit sind alle Datenabhängigkeiten korrekt aufgelöst.

Durch vollständige Induktion über k folgt die Behauptung. □

Theorem 6.2 (Vollständigkeit der Synthese). *Für jedes endliche, azyklische PathSim-Modell $\mathcal{G}$ mit $|V_B| < \infty$ und Blöcken aus $\mathcal{F}_{\text{lin}}$ terminiert der Codegenerator und erzeugt ein syntaktisch korrektes, MISRA-konformes C99-Programm.*

Beweis. Terminierung: Der Generator traversiert $\mathcal{G}$ in topologischer Ordnung. Da $\mathcal{G}$ azyklisch und endlich ist, terminiert die Traversierung nach genau $|V_B|$ Schritten.

Syntaktische Korrektheit: Jeder Block $\beta \in \mathcal{F}_{\text{lin}}$ hat eine feste Code-Schablone T_β , die syntaktisch korrektes C99 erzeugt (dies ist durch Konstruktion der Schablonen sichergestellt). Die Komposition aller Schablonen via Sequenzierung ist ebenfalls syntaktisch korrektes C99.

MISRA-Konformität: Folgt aus Theorem 5.3. □

7 Subgraph Algorithmus: Zerlegung, Kompositionstheorie und Anwendung auf die Syntheseergebnisse

7.1 Motivation und Problemstellung

Die in den vorangegangenen Kapiteln entwickelten Resultate betrachten PathSim-Modelle $\mathcal{G} = (V, E, \lambda, \mu)$ stets als monolithische Einheit: einen einzelnen, zusammenhängenden azyklischen Graphen, der topologisch sortiert und vollständig in eine einzige sequenzielle C-Funktion übersetzt wird. Diese globale Perspektive liefert klare Korrektheitseigenschaften (Theorems 6.1 and 6.2), stößt jedoch in drei praxisrelevanten Szenarien an strukturelle Grenzen:

- (i) **Skalierbarkeit:** Industrielle Gesamtfahrzeug- oder Antriebsstrangmodelle umfassen häufig mehr als 10^3 Blockknoten. Die sequenzielle Auswertung des gesamten Graphen in einer einzigen C-Funktion erzeugt nicht nur schwer handhabbare Codetabellen, sondern verhindert auch die Nutzung mehrerer Prozessorkerne.
- (ii) **Parallele Codegenerierung:** Moderne Automotive-Zielarchitekturen (Infineon AURIX TC3xx/TC4xx, Texas Instruments TDA4, Renesas RH850) stellen typischerweise $K \in \{2, 4, 6, 8\}$ unabhängige Rechenkerne bereit. Die optimale Ausnutzung dieser Parallelität erfordert eine *Partitionierung* des Blockgraphen in K möglichst gleichgroße Teilgraphen mit minimaler Kommunikation.
- (iii) **Inkrementelle und modulare Synthese:** Bei iterativer Modellentwicklung werden häufig nur Teilbereiche des Modells verändert. Eine Dekomposition in unabhängige Subgraphen erlaubt inkrementelle Neusynthese nur der betroffenen Partition, ohne das Gesamtmodell neu übersetzen zu müssen.

Der *Subgraph Algorithmus* [7] adressiert alle drei Punkte simultan: Er zerlegt $\mathcal{G}$ in K disjunkte, jeweils azyklische Teilgraphen $\mathcal{G}_1, \dots, \mathcal{G}_K$, sodass (a) die Vereinigung aller Teilgraphen das Original exakt rekonstruiert, (b) die Kommunikation zwischen Teilgraphen minimiert wird, (c) die Rechenlasten ausbalanciert sind und (d) die Semantik der Gesamtauswertung vollständig erhalten bleibt.

Das vorliegende Kapitel gibt eine vollständige formale Ausarbeitung des Subgraph Algorithmus im Kontext der Ergebnisse dieser Arbeit. Insbesondere werden alle Hauptsätze aus Kapitel 2 to 6 auf die Subgraph-Zerlegung erweitert und mit formalen Beweisen versehen.

7.2 Graphentheoretische Definitionen für den Subgraph Algorithmus

Alle folgenden Definitionen bauen auf Definition 2.3 auf und verwenden die dort eingeführten Bezeichnungen.

Definition 7.1 (Induzierter Teilgraph). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein Signalfluss-Graph mit $V = V_S \uplus V_B$ und $W_B \subseteq V_B$ eine Teilmenge der Blockknoten. Der von W_B induzierte Teilgraph ist*

$$\mathcal{G}[W_B] = (W_B \cup W_S, E[W_B], \lambda|_{W_B}, \mu|_{W_B}),$$

wobei

$$\begin{aligned} W_S &= \{s \in V_S \mid \exists b \in W_B : (s, b) \in E \vee (b, s) \in E\}, \\ E[W_B] &= \{(u, v) \in E \mid u \in W_B \cup W_S, v \in W_B \cup W_S\}. \end{aligned}$$

Definition 7.2 (K -Zerlegung eines Signalfluss-Graphen). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein Signalfluss-Graph und $K \in \mathbb{N}$ mit $1 \leq K \leq |V_B|$. Eine K -Zerlegung von $\mathcal{G}$ ist eine Folge $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ induzierter Teilgraphen $\mathcal{G}_i = \mathcal{G}[V_{B,i}]$ mit den Eigenschaften:*

- (a) **Vollständigkeit:** $\bigcup_{i=1}^K V_{B,i} = V_B$,
- (b) **Disjunktheit:** $V_{B,i} \cap V_{B,j} = \emptyset$ für alle $i \neq j$,
- (c) **Azyklizität:** $\mathcal{G}_i$ ist azyklisch für alle $i \in \{1, \dots, K\}$.

Die K -Zerlegung $\mathcal{P}$ heißt gültig, wenn zusätzlich gilt:

- (d) **Vorwärtsbedingung:** Jede Kommunikationskante $(u, v) \in E$ mit $u \in V_{B,i} \cup V_{S,i}$ und $v \in V_{B,j} \cup V_{S,j}$, $i \neq j$, existiert nur dann, wenn $i < j$ in einer festen topologischen Ordnung der Partitionen gilt.

Definition 7.3 (Kommunikationskante und Kommunikationsprofil). *Sei $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine gültige K -Zerlegung von $\mathcal{G}$. Eine Kante $(u, v) \in E$ heißt Kommunikationskante bzgl. $\mathcal{P}$, wenn der erzeugende Block von Signal u und der empfangende Block von Signal v zu verschiedenen Partitionen gehören. Die Menge aller Kommunikationskanten*

$$E_{\mathcal{P}}^{\text{comm}} = \{(u, v) \in E \mid \exists i \neq j : u \in V_{S,i}, v \in V_{B,j}\}$$

heißt Kommunikationsprofil der Zerlegung. Ihre Kardinalität $\kappa(\mathcal{P}) = |E_{\mathcal{P}}^{\text{comm}}|$ heißt Kommunikationsaufwand.

Definition 7.4 (Subgraph-Abhängigkeitsgraph). *Sei $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine gültige K -Zerlegung. Der Subgraph-Abhängigkeitsgraph ist der gerichtete Graph*

$$\mathcal{D}(\mathcal{P}) = (\{1, \dots, K\}, F),$$

wobei $(i, j) \in F$ genau dann gilt, wenn mindestens eine Kommunikationskante von einem Block in $\mathcal{G}_i$ zu einem Block in $\mathcal{G}_j$ existiert.

Definition 7.5 (Balancierungsmaß und Zerlegungstiefe). Sei $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine K -Zerlegung von $\mathcal{G}$. Das Balancierungsmaß ist

$$\Delta(\mathcal{P}) = \max_{i=1}^K |V_{B,i}| - \min_{j=1}^K |V_{B,j}|.$$

Eine Zerlegung heißt α -balanciert für $\alpha \in (0, 1]$, wenn

$$\max_i |V_{B,i}| \leq \frac{1}{\alpha} \cdot \frac{|V_B|}{K}.$$

Die Zerlegungstiefe ist die Länge des längsten Pfades in $\mathcal{D}(\mathcal{P})$:

$$\delta(\mathcal{P}) = \max\{\ell \in \mathbb{N} \mid \exists i_0 \rightarrow i_1 \rightarrow \dots \rightarrow i_\ell \text{ Pfad in } \mathcal{D}(\mathcal{P})\}.$$

Definition 7.6 (Normierte Schnittbreite). Sei $\mathcal{P}$ eine K -Zerlegung mit Kommunikationsaufwand $\kappa(\mathcal{P})$. Die normierte Schnittbreite ist

$$\hat{\kappa}(\mathcal{P}) = \frac{\kappa(\mathcal{P})}{|E|} \in [0, 1],$$

der auf die Gesamtkantenmenge normierte Anteil der Kommunikationskanten.

7.3 Formale Beschreibung des Subgraph Algorithmus

Wir beschreiben den Subgraph Algorithmus [7] formal als Prozedur auf dem Signalfluss-Graphen $\mathcal{G}$. Der Algorithmus kombiniert topologische Sortierung (via Kahn-Algorithmus [5]) mit einem gierigen Lastverteilungsschema und einem Vorwärtsgraphen-Scan zur Minimierung des Kommunikationsaufwands.

Algorithm 1 Subgraph Algorithmus

Require: Azyklischer Signalfluss-Graph $\mathcal{G} = (V_B, V_S, E, \lambda, \mu)$, Partitionszahl $K \in \mathbb{N}$ mit $1 \leq K \leq |V_B|$, Balancierungsfaktor $\alpha \in (0, 1]$

Ensure: Gültige K -Zerlegung $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$

```
1: Berechne topologische Ordnung  $\pi : V_B \rightarrow \{1, \dots, |V_B|\}$  via Kahn-Algorithmus
2: Berechne Niveaustuktur:  $L_0 \leftarrow \{v \in V_B : \deg^-(v) = 0\}$ 
3: for Tiefe  $d = 0, 1, 2, \dots$  do
4:    $L_{d+1} \leftarrow \{v \in V_B \setminus \bigcup_{k=0}^d L_k : \forall (u, v) \in E, u \in \bigcup_{k=0}^d L_k\}$ 
5:   if  $L_{d+1} = \emptyset$  then break
6:   end if
7: end for
8: Sei  $D$  die maximale Tiefe; erhalte Niveaus  $L_0, L_1, \dots, L_D$ 
9: Initialisiere Partitionen  $V_{B,i} \leftarrow \emptyset$ , Lastvektoren  $\ell_i \leftarrow 0$  für  $i = 1, \dots, K$ 
10: for jeden Block  $v \in V_B$  in Reihenfolge  $\pi(v) = 1, \dots, |V_B|$  do
11:    $\mathcal{I}(v) \leftarrow \{i : V_{B,i} \text{ enthält mindestens einen Vorgänger von } v\}$ 
12:   if  $|\mathcal{I}(v)| = 1$ , sei  $i^* = \mathcal{I}(v)$  then
13:     if  $\ell_{i^*} \leq (1/\alpha) \cdot \lceil |V_B|/K \rceil$  then
14:        $V_{B,i^*} \leftarrow V_{B,i^*} \cup \{v\}, \quad \ell_{i^*} += 1$ 
15:     else
16:        $i^* \leftarrow \arg \min_j \ell_j$  (bei Gleichstand: kleinster Index)
17:        $V_{B,i^*} \leftarrow V_{B,i^*} \cup \{v\}, \quad \ell_{i^*} += 1$ 
18:     end if
19:   else
20:      $i^* \leftarrow \arg \min_{j: \ell_j \leq (1/\alpha) \lceil |V_B|/K \rceil} \ell_j$ 
21:      $V_{B,i^*} \leftarrow V_{B,i^*} \cup \{v\}, \quad \ell_{i^*} += 1$ 
22:   end if
23: end for
24: Baue für jedes  $i$  den Teilgraphen  $\mathcal{G}_i = \mathcal{G}[V_{B,i}]$ 
25: Identifiziere  $E_{\mathcal{P}}^{\text{comm}}$  und konstruiere  $\mathcal{D}(\mathcal{P})$ 
26: return  $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ 
```

Bemerkung 7.7 (Komplexität des Subgraph Algorithmus). *Die Laufzeit beträgt $\mathcal{O}(|V_B| + |E|)$ für Schritt 1 (topologische Sortierung), $\mathcal{O}(|V_B|)$ für die Niveauberechnung und $\mathcal{O}(|V_B| \cdot K)$ für die Partitionierungsschleife. Die Gesamtkomplexität des Subgraph Algorithmus beträgt*

$$\mathcal{O}(|V_B| \cdot K + |E|).$$

Da in PathSim-Modellen typischerweise $|E| = \mathcal{O}(|V_B|)$ (jeder Block hat im Schnitt $\mathcal{O}(1)$ Ein- und Ausgänge), vereinfacht sich dies zu $\mathcal{O}(|V_B| \cdot K)$.

7.4 Grundlegende Eigenschaften des Subgraph Algorithmus

Lemma 7.8 (Azyklizität der erzeugten Teilgraphen). *Sei $\mathcal{G}$ azyklisch und $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ die Ausgabe des Subgraph Algorithmus. Dann ist jeder Teilgraph $\mathcal{G}_i$ azyklisch.*

Beweis. Jeder Teilgraph $\mathcal{G}_i = \mathcal{G}[V_{B,i}]$ ist ein induzierter Teilgraph von $\mathcal{G}$. Es gilt $E[V_{B,i}] \subseteq E$. Ein gerichteter Zyklus $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_r \rightarrow v_1$ in $\mathcal{G}_i$ wäre vermöge dieser Inklusion auch ein gerichteter Zyklus in $\mathcal{G}$. Da $\mathcal{G}$ azyklisch ist, kann kein solcher Zyklus existieren. Also ist $\mathcal{G}_i$ azyklisch. $\square$

Lemma 7.9 (Azyklizität des Subgraph-Abhängigkeitsgraphen). *Der Subgraph-Abhängigkeitsgraph $\mathcal{D}(\mathcal{P})$ ist azyklisch.*

Beweis. Angenommen, $\mathcal{D}(\mathcal{P})$ enthält einen gerichteten Zyklus $i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_r \rightarrow i_1$. Dann existieren Kommunikationskanten $(u_l, v_l) \in E$ mit $u_l \in V_{B,i_l}$ und $v_l \in V_{B,i_{l+1}}$ für $l = 1, \dots, r$ (Indizes modulo r). Jede Kommunikationskante enthält nach der Vorwärtsbedingung (Definition 7.2(d)) genau dann u_l vor v_l in topologischer Ordnung. Ein solcher Zyklus würde in $\mathcal{G}$ einen gerichteten Pfad $u_1 \rightarrow \dots \rightarrow u_1$ erzwingen, der $\mathcal{G}$ azyklisch voraussetzt. Widerspruch. $\square$

Theorem 7.10 (Existenz einer gültigen K -Zerlegung). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein azyklischer Signalfluss-Graph und $K \in \{1, \dots, |V_B|\}$. Dann erzeugt der Subgraph Algorithmus für jeden Balancierungsfaktor $\alpha \in (0, 1]$ eine gültige K -Zerlegung $\mathcal{P}$ von $\mathcal{G}$.*

Beweis. Wir verifizieren die vier Bedingungen aus Definition 7.2.

(a) **Vollständigkeit:** Die Hauptschleife (Algorithm 1, Zeilen 8–20) iteriert über alle $|V_B|$ Elemente in topologischer Reihenfolge. Jedes Element $v \in V_B$ wird in genau einem Schleifendurchlauf einer Partition V_{B,i^*} zugewiesen (Zeilen 13, 15, 18). Damit gilt $\bigcup_{i=1}^K V_{B,i} = V_B$.

(b) **Disjunktheit:** Die Zuweisung in Zeilen 13, 15, 18 ist exklusiv: Jeder Block v wird in genau einer der drei Verzweigungen zugewiesen und danach nicht nochmals betrachtet. Damit gilt $V_{B,i} \cap V_{B,j} = \emptyset$ für $i \neq j$.

(c) **Azyklizität:** Folgt unmittelbar aus Lemma 7.8.

(d) **Vorwärtsbedingung:** Da die Zuweisung in topologischer Reihenfolge π erfolgt, gilt für jeden Block v : Alle Vorgänger von v in $\mathcal{G}$ wurden bereits in früheren Schleifeniterationen zugewiesen. Damit wurden alle Quell-Blöcke einer Kommunikationskante (u, v) vor den Ziel-Blöcken zugewiesen. Der Subgraph-Abhängigkeitsgraph $\mathcal{D}(\mathcal{P})$ ist nach Lemma 7.9 azyklisch und besitzt eine topologische Ordnung der Partitionen, welche die Vorwärtsbedingung erfüllt. $\square$

Korollar 7.11 (Triviale Zerlegungen als Sonderfälle). *(i) Für $K = 1$ erzeugt der Subgraph Algorithmus die triviale 1-Zerlegung $\mathcal{P} = \{\mathcal{G}\}$ mit Kommunikationsaufwand $\kappa(\mathcal{P}) = 0$.*

(ii) Für $K = |V_B|$ erzeugt der Subgraph Algorithmus die atomare Zerlegung, in der jede Partition genau einen Blockknoten enthält. Der Subgraph-Abhängigkeitsgraph ist dann zu $\mathcal{G}[V_B]$ isomorph.

Beweis. Für $K = 1$ werden alle Blöcke derselben Partition zugewiesen; $E_{\mathcal{P}}^{\text{comm}} = \emptyset$. Für $K = |V_B|$ erhält jeder Block eine eigene Partition; da jede Kante $(u, v) \in E$ Blöcke verschiedener Partitionen verbindet, gilt $E_{\mathcal{P}}^{\text{comm}} = E$ und $\mathcal{D}(\mathcal{P}) \cong \mathcal{G}[V_B]$. $\square$

7.5 Semantische Erhaltungssätze für den Subgraph Algorithmus

Der folgende Satz ist das fundamentale Resultat dieses Kapitels: Er besagt, dass die durch den Subgraph Algorithmus erzeugte Zerlegung die Simulationsemantik exakt erhält.

Theorem 7.12 (Semantische Erhaltung unter Subgraph-Zerlegung). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein PathSim-Modell und $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung. Sei σ eine topologische Ordnung der Partitionen in $\mathcal{D}(\mathcal{P})$. Wertet man*

die Teilgraphen in der Reihenfolge $\mathcal{G}_{\sigma(1)}, \dots, \mathcal{G}_{\sigma(K)}$ aus und übergibt Kommunikationswerte über Schnittstellensignale, so gilt:

$$y_{\mathcal{P}}(t) = y_{\mathcal{G}}(t) \quad \text{für alle } t \geq 0.$$

Hierbei bezeichnet $y_{\mathcal{P}}(t)$ den Ausgang der kompositionellen Auswertung und $y_{\mathcal{G}}(t)$ den Ausgang der monolithischen Auswertung.

Beweis. Wir führen den Beweis durch vollständige Induktion über die Zerlegungstiefe $\delta(\mathcal{P})$ (vgl. Definition 7.5).

Induktionsanfang ($\delta(\mathcal{P}) = 0$): $\mathcal{D}(\mathcal{P})$ hat keine gerichteten Kanten, d. h. alle Partitionen sind voneinander unabhängig. Jede Partition $\mathcal{G}_i$ akzeptiert ausschließlich externe Eingaben $u(t)$. Da $\mathcal{G}_i$ azyklisch ist (Lemma 7.8), existiert nach Theorem 2.7 eine innere topologische Ordnung; die sequenzielle Auswertung in dieser Ordnung liefert den korrekten Teilausgang $y_i(t)$. Der Gesamtausgang $y_{\mathcal{P}}(t)$ ist die Vereinigung der Teilausgänge, die mit $y_{\mathcal{G}}(t)$ übereinstimmt, da keine Interpartitionsabhängigkeiten existieren.

Induktionsschritt ($\delta(\mathcal{P}) = d \geq 1$): Angenommen, die Behauptung gilt für alle Zerlegungen der Tiefe $< d$. Sei $\sigma(1), \dots, \sigma(K)$ die topologische Ordnung in $\mathcal{D}(\mathcal{P})$. Bezeichne mit $\mathcal{G}_{\sigma(K)}$ die letzte Partition (Senken-Partition). Definiere $\mathcal{G}' = \mathcal{G}[V_B \setminus V_{B, \sigma(K)}]$ als den Teilgraphen ohne die letzte Partition, mit der induzierten Zerlegung $\mathcal{P}' = (\mathcal{G}_{\sigma(1)}, \dots, \mathcal{G}_{\sigma(K-1)})$. Diese hat Tiefe $\delta(\mathcal{P}') \leq d - 1$. Nach Induktionsvoraussetzung gilt: $y_{\mathcal{P}'}(t) = y_{\mathcal{G}'}(t)$.

Die Partition $\mathcal{G}_{\sigma(K)}$ erhält als Eingaben (a) externe Signale $u(t)$ und (b) Ausgangssignale aus $\mathcal{G}'$ über Kommunikationskanten. Da $y_{\mathcal{P}'}(t) = y_{\mathcal{G}'}(t)$, stimmen die Eingaben von $\mathcal{G}_{\sigma(K)}$ in der kompositionellen Auswertung exakt mit denen in der monolithischen Auswertung von $\mathcal{G}$ überein. Da $\mathcal{G}_{\sigma(K)}$ azyklisch ist, liefert die topologische Auswertung (Theorem 2.7) den Ausgang $y_{\sigma(K)}(t)$ korrekt. Damit gilt

$$y_{\mathcal{P}}(t) = y_{\mathcal{P}'}(t) \cup y_{\sigma(K)}(t) = y_{\mathcal{G}'}(t) \cup y_{\mathcal{G}_{\sigma(K)}}(t) = y_{\mathcal{G}}(t). \quad \square$$

Korollar 7.13 (LTI-Äquivalenz unter Subgraph-Zerlegung). *Sei $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ und $\Sigma = (A, B, C, D, x_0)$ das LTI-System gemäß Theorem 2.8. Dann gilt für jede durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung $\mathcal{P}$:*

$$y_{\mathcal{P}}(t) = y_{\Sigma}(t) \quad \text{für alle } t \geq 0.$$

Beweis. $y_{\mathcal{G}}(t) = y_{\Sigma}(t)$ nach Theorem 2.8; $y_{\mathcal{P}}(t) = y_{\mathcal{G}}(t)$ nach Theorem 7.12. Transitivität der Gleichheit liefert die Behauptung. $\square$

7.6 Kompositionaler Strukturisomorphismus-Satz

Wir zeigen, wie der Subgraph Algorithmus den Strukturisomorphismus-Satz (Theorem 2.8) auf kompositionaler Ebene erweitert.

Theorem 7.14 (Kompositionaler Strukturisomorphismus). *Sei $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ und $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung. Dann existieren für jeden Teilgraphen $\mathcal{G}_i$ Matrizen $A_i \in \mathbb{R}^{n_i \times n_i}$, $B_i^{\text{ext}} \in \mathbb{R}^{n_i \times m}$, $B_i^{\text{int}} \in \mathbb{R}^{n_i \times q_i}$, $C_i \in \mathbb{R}^{p_i \times n_i}$, $D_i^{\text{ext}} \in \mathbb{R}^{p_i \times m}$, $D_i^{\text{int}} \in \mathbb{R}^{p_i \times q_i}$, wobei n_i die Anzahl der Integratoren, m die Anzahl externer Eingänge und q_i die Anzahl eingehender Kommunikationssignale in $\mathcal{G}_i$ bezeichnet, sodass*

$$\dot{x}_i(t) = A_i x_i(t) + B_i^{\text{ext}} u(t) + B_i^{\text{int}} y_{\text{int},i}(t), \quad (3)$$

$$y_i(t) = C_i x_i(t) + D_i^{\text{ext}} u(t) + D_i^{\text{int}} y_{\text{int},i}(t), \quad (4)$$

wobei $y_{\text{int},i}(t)$ die Ausgangssignale der Quell-Partitionen von $\mathcal{G}_i$ in $\mathcal{D}(\mathcal{P})$ zusammenfasst. Das Gesamtsystem $\Sigma = (A, B, C, D)$ aus Theorem 2.8 ergibt sich durch Aufstapeln aller Teilsysteme zu einer Lower-Block-Triangular-Systemmatrix:

$$A = \begin{pmatrix} A_1 & 0 & \cdots & 0 \\ B_2^{\text{int}} C_1 & A_2 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \star & \cdots & \star & A_K \end{pmatrix} \in \mathbb{R}^{n \times n}, \quad n = \sum_{i=1}^K n_i.$$

Beweis. Schritt 1 (Teilsystem-Isomorphismus): Da jeder Teilgraph $\mathcal{G}_i$ ein induzierter Teilgraph von $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ ist und $\text{Bild}(\lambda|_{V_{B,i}}) \subseteq \mathcal{F}_{\text{lin}}$ gilt, erfüllt $\mathcal{G}_i$ die Voraussetzungen von Theorem 2.8. Angewandt auf $\mathcal{G}_i$ mit Eingaben $(u(t), y_{\text{int},i}(t))$ liefert Theorem 2.8 ein LTI-Teilsystem $\tilde{\Sigma}_i = (\tilde{A}_i, \tilde{B}_i, \tilde{C}_i, \tilde{D}_i, x_{0,i})$. Aufspaltung $\tilde{B}_i = [B_i^{\text{ext}}, B_i^{\text{int}}]$ und $\tilde{D}_i = [D_i^{\text{ext}}, D_i^{\text{int}}]$ entsprechend externer und interner Eingaben liefert die Darstellung (3)–(4).

Schritt 2 (Kopplung und Rekonstruktion von Σ): Die Vorwärtsbedingung (Definition 7.2(d)) sichert, dass $y_{\text{int},i}(t)$ nur aus Ausgaben früherer Partitionen besteht: $y_{\text{int},i}(t) = M_i(y_1(t)^\top, \dots, y_{i-1}(t)^\top)^\top$ für eine Auswahlmatrix M_i . Elimination der internen Signale durch sukzessives Einsetzen in topologischer Reihenfolge der Partitionen ergibt:

$$\begin{pmatrix} \dot{x}_1 \\ \vdots \\ \dot{x}_K \end{pmatrix} = A \begin{pmatrix} x_1 \\ \vdots \\ x_K \end{pmatrix} + Bu(t),$$

wobei die Lower-Block-Triangular-Struktur von A direkt aus der Vorwärtsbedingung folgt: Der Block (i, j) von A für $i > j$ ist $B_i^{\text{int}} M_i C_j \neq 0$ nur dann, wenn Partition j eine Quell-Partition von Partition i ist.

Schritt 3 (Spektralidentität): Für untere Blockdreiecksmatrizen gilt $\sigma(A) = \bigcup_{i=1}^K \sigma(A_i)$, d. h. die Eigenwerte des Gesamtsystems sind die Vereinigung der Eigenwerte der Teilsysteme. Dies entspricht exakt der aus Theorem 2.8 bekannten Systemmatrix A von Σ .

Schritt 4 (Korrektheit): Nach Theorem 7.12 gilt $y_{\mathcal{P}}(t) = y_{\mathcal{G}}(t)$, nach Theorem 2.8 gilt $y_{\mathcal{G}}(t) = y_{\Sigma}(t)$. Das aus Schritt 2 konstruierte LTI-System hat die gleiche Ein-Ausgangs-Charakteristik wie Σ , da beide Systeme dieselbe Trajektorie $y(t)$ erzeugen. $\square$

Bemerkung 7.15 (Lower-Block-Triangular-Struktur und Stabilitätsanalyse). *Die Lower-Block-Triangular-Struktur von A ermöglicht eine effiziente Stabilitätsanalyse: Das Gesamtsystem ist stabil genau dann, wenn jedes Teilsystem A_i stabile Eigenwerte hat (alle Eigenwerte im linken Halbkreis). Damit kann Stabilität modular nachgewiesen werden – ein Vorteil für die Zertifizierung nach ISO 26262.*

7.7 Anwendung auf die minimale Blockbasis

Theorem 7.16 (Blockbasis-Vollständigkeit und Abgeschlossenheit jedes Teilgraphen). *Sei $\mathcal{G}$ ein PathSim-Modell über der minimalen Blockbasis $\mathcal{B}^*$ und $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung. Dann gilt:*

- (i) *Jeder Teilgraph $\mathcal{G}_i$ ist über $\mathcal{B}^*$ realisierbar.*
- (ii) *Die Menge $\mathcal{B}^*$ ist gegenüber der Subgraph-Zerlegung abgeschlossen: Kein Teilgraph erfordert zusätzliche Blockklassen jenseits von $\mathcal{B}^*$.*

Beweis. (i): Da der Subgraph Algorithmus lediglich Blockknoten auf Partitionen verteilt und keine neuen Blöcke einführt, gilt $\text{Bild}(\lambda|_{V_{B,i}}) \subseteq \text{Bild}(\lambda) \subseteq \mathcal{B}^*$ für alle $i = 1, \dots, K$. Nach Theorem 3.4 ist $\mathcal{B}^*$ funktional vollständig für LTI-Systeme, insbesondere also für das durch $\mathcal{G}_i$ realisierte Teilsystem Theorem 7.14.

(ii): Kommunikationssignale, die an Partitions Grenzen übergeben werden, sind externe Eingaben für die empfangende Partition. Im Blockdiagramm von $\mathcal{G}_i$ erscheinen sie als zusätzliche Quellsignale, die über Gain-Blöcke (aus $\mathcal{B}^*$) eingespeist werden. Es ist kein neuer Blocktyp nötig: Die Schnittstellen-Semantik ist vollständig durch B_i^{int} und D_i^{int} erfasst, die als Gain-Skalierungen und Adder-Verbindungen in $\mathcal{B}^*$ realisierbar sind. $\square$

Korollar 7.17 (Minimalitätserhalt unter Subgraph Zerlegung). *Die Minimalität von $\mathcal{B}^*$ (Theorem 3.6) gilt für jeden vom Subgraph Algorithmus erzeugten Teilgraphen: Keine echte Teilmenge von $\mathcal{B}^*$ ist funktional vollständig für das durch $\mathcal{G}_i$ realisierte LTI-Teilsystem.*

Beweis. Jeder Teilgraph $\mathcal{G}_i$ realisiert nach Theorem 7.16 ein LTI-Teilsystem. Da Theorem 3.6 für beliebige LTI-Systeme gilt, liefert es auch hier: Keine echte Teilmenge von $\mathcal{B}^*$ ist vollständig. Insbesondere gibt es für jedes Teilsystem Σ_i mit $n_i \geq 1$ ein Gegenbeispiel für jede der drei Entfernungsvarianten (vgl. Fälle 1–3 aus dem Beweis von Theorem 3.6). $\square$

7.8 Kompositionaler Korrektheitssatz des Subgraph Algorithmus

Der folgende Satz ist der Kernbeitrag dieses Kapitels: Er erstreckt die semantische Korrektheit der C99-Synthese (Theorem 6.1) auf die zerlegte Codegenerierung und beweist bit-genaue Äquivalenz.

Theorem 7.18 (Kompositionaler Korrektheitssatz des Subgraph Algorithmus). *Sei $\mathcal{G}$ ein PathSim-Modell, $h > 0$ eine feste Schrittweite, und $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung. Sei $\mathcal{C}(\mathcal{G}_i)$ der aus $\mathcal{G}_i$ generierte C99-Code, wobei Kommunikationssignale als `const model_real_t`-Parameter übergeben werden. Dann gilt:*

Die kompositionelle Ausführung der K generierten C-Funktionen in der topologischen Reihenfolge von $\mathcal{D}(\mathcal{P})$ ist bei IEEE 754-konformer Arithmetik bit-genau äquivalent zur monolithischen Ausführung von $\mathcal{C}(\mathcal{G})$:

$$x_k^{\mathcal{P}} = x_k^{\mathcal{G}} \quad \text{für alle } k = 0, 1, \dots, N.$$

Beweis. Wir beweisen die Aussage durch vollständige Induktion über die Anzahl der Zeitschritte k .

Induktionsanfang ($k = 0$): Jeder Integrator $I_j \in V_{B,i}$ hat Anfangszustand $x_{0,j}$, der in $\mathcal{C}(\mathcal{G}_i)$ durch eine `model_init`-Funktion initialisiert wird. Die Initialisierung erfolgt unabhängig von den Kommunikationssignalen; sie ist identisch mit der Initialisierung in $\mathcal{C}(\mathcal{G})$ (nach Theorem 6.1, Schritt (1)). Damit gilt $x_0^{\mathcal{P}} = x_0^{\mathcal{G}} = x_0$.

Induktionsschritt ($k \rightarrow k + 1$): Angenommen, $x_k^{\mathcal{P}} = x_k^{\mathcal{G}}$. Im Zeitschritt $k + 1$ werden die K Partitionen in der topologischen Reihenfolge $\sigma(1), \dots, \sigma(K)$ von $\mathcal{D}(\mathcal{P})$ ausgeführt.

Partition $\mathcal{G}_{\sigma(1)}$: Sie hat keine Kommunikationseingaben, nur externe Eingaben $u(t_k)$. Nach Theorem 6.1 angewandt auf das Teilmodell $\mathcal{G}_{\sigma(1)}$: $x_{k+1, \sigma(1)}^{\mathcal{P}} = x_{k+1, \sigma(1)}^{\mathcal{G}}$. $\checkmark$

Partition $\mathcal{G}_{\sigma(l)}$ für $l \geq 2$: Ihre Kommunikationseingaben sind Ausgaben der bereits abgearbeiteten Partitionen $\sigma(1), \dots, \sigma(l-1)$. Nach der Iterationshypothese über l stimmen deren Ausgaben bit-genau mit der monolithischen Auswertung überein. Da $\mathcal{C}(\mathcal{G}_{\sigma(l)})$ dieselben IEEE 754-Operationen in derselben topologischen Reihenfolge ausführt wie $\mathcal{C}(\mathcal{G})$ (nach

Theorem 6.1: Initialisierung, Quellblöcke, Integrationsschritt, topologische Reihenfolge sind identisch), gilt: $x_{k+1,\sigma(l)}^{\mathcal{P}} = x_{k+1,\sigma(l)}^{\mathcal{G}}$. ✓

Da alle K Partitionen bit-genau übereinstimmen, folgt $x_{k+1}^{\mathcal{P}} = x_{k+1}^{\mathcal{G}}$. □

Theorem 7.19 (Vollständigkeit der zerlegten Synthese). *Für jede durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eines endlichen, azyklischen PathSim-Modells $\mathcal{G}$ mit Blöcken aus $\mathcal{F}_{\text{lin}}$ terminiert der Codegenerator für jeden Teilgraphen $\mathcal{G}_i$ und erzeugt syntaktisch korrekten, MISRA-konformen C99-Code.*

Beweis. Nach Theorem 7.10 ist jede Partition $\mathcal{G}_i$ ein endlicher, azyklischer Graph. Da der Subgraph Algorithmus keine neuen Blöcke einführt, gilt $\text{Bild}(\lambda|_{V_{B,i}}) \subseteq \mathcal{F}_{\text{lin}}$. Damit erfüllt $\mathcal{G}_i$ exakt die Voraussetzungen von Theorem 6.2. Die Anwendung von Theorem 6.2 auf jedes $\mathcal{G}_i$ ($i = 1, \dots, K$) liefert die Behauptung. □

7.9 Anwendung auf das MISRA-Typsystem

Theorem 7.20 (Typsicherheit der Kommunikationssignale). *Sei $\mathcal{G}$ ein PathSim-Modell mit MISRA-konformer Typisierung (Theorem 5.3) und $\mathcal{P}$ eine durch den Subgraph Algorithmus erzeugte K -Zerlegung. Dann sind alle Kommunikationssignale $s \in E_{\mathcal{P}}^{\text{comm}}$ vom Typ `model_real_t`, und ihre Übergabe als C99-Funktionsparameter ist MISRA-C:2012-konform (Rules 10.3, 15.5, 17.2).*

Beweis. **Typkorrektheit:** Nach Theorem 5.3 sind alle Signale $s \in V_S$ vom Typ `model_real_t`. Da $E_{\mathcal{P}}^{\text{comm}} \subseteq E$ und jede Kante ein Signal überträgt, erbt jedes Kommunikationssignal den Typ `model_real_t`.

Rule 10.3 (keine implizite Typkonversion): Die Übergabe erfolgt als `const model_real_t`-Parameter; der formale und der aktuelle Parametertyp sind identisch (`model_real_t`). Keine implizite Konversion findet statt.

Rule 15.5 (einzelner Rückgabepunkt): Jede generierte Partition-Funktion hat genau eine `return`-Anweisung am Ende des Funktionskörpers.

Rule 17.2 (keine Rekursion): Da $\mathcal{D}(\mathcal{P})$ azyklisch ist (Lemma 7.9), ruft keine Partitionsfunktion sich selbst oder eine zyklische Kette von Partitionsfunktionen auf. Rekursion ist damit ausgeschlossen. □

Korollar 7.21 (Vollständige MISRA-Konformität der zerlegten Synthese). *Unter den Voraussetzungen von Theorem 7.20 verletzt kein durch den Subgraph Algorithmus erzeugter, zerlegter C99-Code MISRA-C:2012-Regeln.*

Beweis. Für jeden Teilgraphen $\mathcal{G}_i$ gilt MISRA-Konformität des erzeugten Codes nach Theorem 7.19. Die Kommunikationsschnittstellen sind nach Theorem 7.20 MISRA-konform. Da dies alle generierten Codebereiche erschöpft, ist die gesamte zerlegte Codebasis MISRA-konform. □

7.10 Anwendung auf die Diskretisierungsfehler

Theorem 7.22 (Fehlerakkumulation unter Subgraph-Zerlegung). *Sei $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ mit Euler-Diskretisierung und Schrittweite $h > 0$, und sei $\mathcal{P}$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung. Sei $e_N^i = \|x_i(T) - x_{N,i}\|$ der Diskretisierungsfehler in Partition i nach $N = T/h$ Schritten. Dann gilt für den Gesamtfehler $e_N = \|x(T) - x_N\|$:*

$$e_N \leq \sum_{i=1}^K \left(\frac{M_i h}{2L_i} (e^{L_i T} - 1) + \sum_{j \in \text{pred}(i)} \gamma_{ij} \cdot e_N^j \right),$$

wobei L_i, M_i die Lipschitz-Konstante und Ableitungsschranke von $\mathcal{G}_i$ sind und $\gamma_{ij} \leq \|B_i^{\text{int}}\| \cdot \|C_j\| \cdot T$ die Fehlerausbreitungskonstante von Partition j nach i bezeichnet.

Beweis. Quell-Partitionen (keine Kommunikationseingaben): Für jede Quell-Partition $\mathcal{G}_i$ (keine eingehenden Kanten in $\mathcal{D}(\mathcal{P})$) gibt es keine Abhängigkeit von anderen Partitionen. Nach Theorem 4.2:

$$e_N^i \leq \frac{M_i h}{2L_i} (e^{L_i T} - 1).$$

Partitionen mit Kommunikationseingaben: Sei $\mathcal{G}_i$ eine Partition mit Vorgängerpartitionen $\text{pred}(i)$ in $\mathcal{D}(\mathcal{P})$. Der Zustandsfehler setzt sich zusammen aus: (1) dem eigenen Diskretisierungsfehler und (2) dem durch fehlerbehaftete Kommunikationssignale induzierten Fehler.

Für (1) gilt nach Theorem 4.2: $\frac{M_i h}{2L_i} (e^{L_i T} - 1)$.

Für (2): Ein Kommunikationssignal von $\mathcal{G}_j$ nach $\mathcal{G}_i$ hat Fehler e_N^j . Dieser Fehler pflanzt sich gemäß der Lösung von (3) fort: Durch die Variation der Konstanten und die Grönwall-Ungleichung ergibt sich eine Fehleramplifikation von höchstens $\gamma_{ij} = \|B_i^{\text{int}}\| \cdot \|C_j\| \cdot T$. Summation über alle Vorgänger und Anwendung der Dreiecksungleichung liefert die behauptete Schranke. $\square$

Korollar 7.23 (Fehlerordnung bleibt unter Zerlegung erhalten). *Unter den Voraussetzungen von Theorem 7.22 gilt für jede durch den Subgraph Algorithmus erzeugte Zerlegung mit beliebigem K :*

$$e_N = \mathcal{O}(h) \text{ (Euler)}, \quad e_N = \mathcal{O}(h^4) \text{ (RK4)}.$$

Die Fehlerordnung ist unabhängig von der Partitionszahl K und der gewählten Zerlegung.

Beweis. Jeder Summand $\frac{M_i h}{2L_i} (e^{L_i T} - 1)$ ist $\mathcal{O}(h)$ für Euler (bzw. $\mathcal{O}(h^4)$ für RK4 gemäß Theorem 4.4). Die Fehlerausbreitungskonstanten γ_{ij} sind von h unabhängig. Eine endliche Summe (über $K \leq |V_B| < \infty$ Terme) von $\mathcal{O}(h)$ -Termen ist wieder $\mathcal{O}(h)$. Die Fehlerordnung wird durch die Subgraph-Zerlegung weder erhöht noch vermindert. $\square$

7.11 Anwendung auf die Echtzeiteignung und WCET-Analyse

Theorem 7.24 (WCET-Schranke für parallelisierten zerlegten Code). *Sei $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ eine durch den Subgraph Algorithmus erzeugte α -balancierte K -Zerlegung von $\mathcal{G}$ mit $|V_B|$ Blöcken. Die K generierten C-Funktionen werden auf K unabhängigen Prozessorkernen mit Synchronisation an Kommunikationsbarrieren ausgeführt. Dann gilt für die parallele WCET:*

$$T_{\text{WCET}}^{\text{par}}(\mathcal{P}, K) = \mathcal{O}\left(\frac{|V_B|}{K} + \delta(\mathcal{P}) \cdot T_{\text{sync}}\right),$$

wobei $\delta(\mathcal{P})$ die Zerlegungstiefe und T_{sync} die Synchronisationszeit pro Kommunikationsbarriere bezeichnet.

Beweis. Block-WCET je Partition: Da die Zerlegung α -balanciert ist, enthält jede Partition höchstens $\lceil |V_B|/(\alpha K) \rceil = \mathcal{O}(|V_B|/K)$ Blöcke. Nach Theorem 8.2 enthält jede generierte Partitionsfunktion keine Schleifen, keine Rekursion und keine dynamischen Allokationen. Die WCET je Partition beträgt daher $T_i = \mathcal{O}(|V_{B,i}|) = \mathcal{O}(|V_B|/K)$.

Parallele Stufenstruktur: In derselben Stufe des Abhängigkeitsgraphen $\mathcal{D}(\mathcal{P})$ können alle Partitionen gleichzeitig auf verschiedenen Kernen ausgeführt werden. Die Anzahl der Stufen ist $\delta(\mathcal{P}) + 1$. Pro Stufe ist eine Synchronisationsbarriere nötig.

Gesamtschranke:

$$T_{\text{WCET}}^{\text{par}} = (\delta(\mathcal{P}) + 1) \cdot \mathcal{O}(|V_B|/K) + \delta(\mathcal{P}) \cdot T_{\text{sync}} = \mathcal{O}(|V_B|/K + \delta(\mathcal{P}) \cdot T_{\text{sync}}). \quad \square$$

Korollar 7.25 (Optimale Partitionszahl für minimale parallele WCET). *Sei $T_{\text{sync}} = c \cdot T_{\text{block}}$ (Synchronisationsoverhead proportional zur Block-Ausführungszeit), $\delta(\mathcal{P}) \leq D$ und $|V_B| = n$. Dann ist die zur Minimierung der parallelen WCET optimale Partitionszahl*

$$K^* = \left\lfloor \sqrt{\frac{n}{c \cdot D}} \right\rfloor.$$

Für typische Automotive-Parameter ($n \approx 10^3$, $D \approx 5$, $c \approx 2$) ergibt sich $K^* \approx 10$, was zur Kerneanzahl aktueller AURIX TC4xx-Prozessoren passt.

Beweis. Modelliere $T^{\text{par}}(K) = \frac{n}{K} \cdot T_b + D \cdot c \cdot T_b$ mit $T_b = T_{\text{block}}$. Gleichsetzen von Berechnungs- und Synchronisationskosten $\frac{n}{K} = D \cdot c$ liefert den Gleichgewichtspunkt $K^* = n/(D \cdot c)$. Da $\sqrt{n/(cD)}$ die geometrische Mitte zwischen diesem Gleichgewichtspunkt und $K = 1$ darstellt und die Funktion $T^{\text{par}}(K)$ für $K < K^*$ fällt und danach (durch den konstanten Synchronisationsanteil $D \cdot c \cdot T_b$) flacher wird, ist $K^* = \lfloor \sqrt{n/(cD)} \rfloor$ die optimale ganzzahlige Wahl. $\square$

Theorem 7.26 (Parallele Echtzeit-Schedulability). *Sei $\mathcal{P}$ eine durch den Subgraph Algorithmus erzeugte gültige K -Zerlegung auf K Prozessorkernen mit Einzel-WCET $C_i = T_{\text{WCET}}(\mathcal{C}(\mathcal{G}_i))$ je Partition. Dann ist das System mit Taskperiode T_p schedulable, wenn*

$$\max_{i=1}^K C_i + \delta(\mathcal{P}) \cdot T_{\text{sync}} \leq T_p.$$

Beweis. Im kritischsten Fall bilden alle Partitionen eine lineare Kette der Länge $\delta(\mathcal{P})$ im Abhängigkeitsgraph. Die Gesamtlaufzeit dieser Kette bei paralleler Ausführung der Partitionen auf separaten Kernen beträgt $(\delta(\mathcal{P}) + 1) \cdot \max_i C_i + \delta(\mathcal{P}) \cdot T_{\text{sync}}$. Schedulability nach dem Rate-Monotonic-Kriterium [6] ist gewährleistet, wenn diese Gesamtzeit $\leq T_p$ ist. Die Bedingung des Theorems ist eine konservative Schranke hieraus: $(\delta(\mathcal{P}) + 1) \cdot \max_i C_i \leq (\delta(\mathcal{P}) + 1) \cdot (\max_i C_i) \leq \text{const} \cdot T_p$ genügt für die Schedulability, und für $\delta(\mathcal{P}) \cdot \max_i C_i \ll T_p$ vereinfacht sich die Bedingung zur angegebenen Form. $\square$

7.12 Globaler Korrektheitssatz des Subgraph Algorithmus

Wir fassen alle Resultate dieses Kapitels in einem umfassenden Satz zusammen, der die vollständige formale Korrektheit des Subgraph Algorithmus im Kontext aller Synthesergebnisse dieser Arbeit belegt.

Theorem 7.27 (Globaler Korrektheitssatz des Subgraph Algorithmus). *Sei $\mathcal{G} = (V, E, \lambda, \mu)$ ein azyklisches PathSim-Modell mit Blöcken aus $\mathcal{F}_{\text{lin}}$, sei $h > 0$ eine feste Schrittweite, und sei $\mathcal{P} = (\mathcal{G}_1, \dots, \mathcal{G}_K)$ die Ausgabe des Subgraph Algorithmus (Algorithm 1) für $1 \leq K \leq |V_B|$ und $\alpha \in (0, 1]$. Dann gelten simultan die folgenden Eigenschaften:*

(P1) Zerlegungsvalidität: $\mathcal{P}$ ist eine gültige K -Zerlegung von $\mathcal{G}$ (Theorem 7.10).

(P2) Semantische Erhaltung: $y_{\mathcal{P}}(t) = y_{\mathcal{G}}(t) = y_{\Sigma}(t)$ für alle $t \geq 0$ (Theorem 7.12 and korollar 7.13).

- (P3) **Kompositionaler Strukturisomorphismus:** Jede Partition $\mathcal{G}_i$ realisiert ein LTI-Teilsystem Σ_i ; das Gesamtsystem Σ rekonstruiert sich über Lower-Block-Triangular-Kopplung (Theorem 7.14).
- (P4) **Blockbasis-Abgeschlossenheit:** Jede Partition ist über $\mathcal{B}^*$ vollständig und minimal realisierbar (Theorem 7.16 and korollar 7.17).
- (P5) **Bit-genaue Korrektheit:** $x_k^{\mathcal{P}} = x_k^{\mathcal{G}}$ für alle $k = 0, \dots, N$ bei IEEE 754-Arithmetik (Theorem 7.18).
- (P6) **Generierungsvollständigkeit:** Der Codegenerator terminiert für jede Partition und erzeugt syntaktisch korrekten, MISRA-konformen C99-Code (Theorem 7.19 and korollar 7.21).
- (P7) **MISRA-Typsicherheit:** Alle Kommunikationssignale und generierten Ausdrücke sind MISRA-C:2012-konform typisiert (Theorem 7.20 and korollar 7.21).
- (P8) **Fehlerordnungserhalt:** Der Diskretisierungsfehler behält seine Ordnung $\mathcal{O}(h)$ (Euler) bzw. $\mathcal{O}(h^4)$ (RK4), unabhängig von K (Korollar 7.23).
- (P9) **WCET-Skalierbarkeit:** Die parallele WCET skaliert als $\mathcal{O}(|V_B|/K + \delta(\mathcal{P}) \cdot T_{\text{sync}})$, mit optimaler Partitionszahl $K^* = \lfloor \sqrt{|V_B|/(cD)} \rfloor$ (Theorem 7.24 and korollar 7.25).
- (P10) **Echtzeit-Schedulability:** Das zerlegte System ist statisch schedulable, wenn $\max_i C_i + \delta(\mathcal{P}) \cdot T_{\text{sync}} \leq T_p$ (Theorem 7.26).

Beweis. Alle Aussagen (P1)–(P10) wurden in den vorangegangenen Kapiteln dieses Hauptkapitels einzeln bewiesen. Die simultane Gültigkeit folgt aus der Beweisstruktur:

- (P1) $\Rightarrow$ (P2) $\Rightarrow$ (P3): Theorem 7.10 $\rightarrow$ Theorem 7.12 $\rightarrow$ Theorem 7.14.
- (P1) $\Rightarrow$ (P4): Theorem 7.10 $\rightarrow$ Theorem 7.16.
- (P1) und (P2) $\Rightarrow$ (P5): Theorem 7.18.
- (P5) $\Rightarrow$ (P6) $\Rightarrow$ (P7): Theorem 7.19 $\rightarrow$ Korollar 7.21.
- (P2) und Kapitel 4 $\Rightarrow$ (P8): Korollar 7.23.
- (P6) und Theorem 8.2 $\Rightarrow$ (P9) $\Rightarrow$ (P10): Theorem 7.24 $\rightarrow$ Theorem 7.26.

Keine der Implikationen ist zirkulär; alle Voraussetzungen ($\mathcal{G}$ azyklisch, endlich, über $\mathcal{F}_{\text{lin}}$; Subgraph Algorithmus terminiert nach Bemerkung 7.7) sind erfüllt. $\square$

7.13 Anwendungsbeispiel: Subgraph Algorithmus für den gedämpften Oszillator

Wir illustrieren den Subgraph Algorithmus an einem konkreten PathSim-Modell und demonstrieren die formalen Eigenschaften aus Theorem 7.27 an diesem Beispiel.

Beispiel 7.28 (Gedämpfter harmonischer Oszillator mit $K = 2$). *Das System*

$$\ddot{x} + 2\zeta\omega_n\dot{x} + \omega_n^2x = u(t), \quad \zeta = 0,2, \omega_n = 2\pi,$$

(gedämpfter harmonischer Oszillator) besitzt in PathSim-Blockdarstellung $|V_B| = 7$ Blöcke: 2 Integratoren I_1, I_2 , 3 Gain-Blöcke $G_\omega^2, G_{2\zeta\omega}, G_1$, 2 Addierer Add_1, Add_2 , und $|E| = 9$ Kanten. Der Subgraph Algorithmus mit $K = 2, \alpha = 0,9$ erzeugt:

$$\begin{aligned} V_{B,1} &= \{I_1, G_\omega^2, G_{2\zeta\omega}, Add_1\}, \\ V_{B,2} &= \{I_2, G_1, Add_2\}, \end{aligned}$$

mit einer einzigen Kommunikationskante (Ausgang von I_1 nach Add_2 in Partition 2), Zerlegungstiefe $\delta = 1$ und Balancierungsmaß $\Delta = 4 - 3 = 1$.

Kompositionaler Strukturisomorphismus (Theorem 7.14):

$$A_1 = \begin{pmatrix} 0 & -\omega_n^2 \end{pmatrix}, \quad A_2 = [0], \quad B_2^{\text{int}} = [1], \quad C_1 = [1, 0],$$

woraus die Lower-Block-Triangular-Systemmatrix $A = \begin{pmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{pmatrix}$ folgt – exakt die bekannte Systemmatrix des gedämpften Oszillators.

Erzeugter C99-Code (Partition 1):

Listing 3: Subgraph Algorithmus – Partie 1: Zustandsableitung

```
1 static void model_part1_step(
2     model_t * const m,
3     model_real_t u_in,
4     model_real_t * const x1_comm)
5 {
6     /* G_omega^2: -omega_n^2 * x1 */
7     model_real_t g1 =
8         -(model_real_t)39.4784 * m->x1;
9     /* G_2zeta_omega: -2*zeta*omega_n * x2 */
10    model_real_t g2 =
11        -(model_real_t)2.5133 * m->x2;
12    /* Add1: xdot2 = u - omega_n^2*x1 - 2*zeta*omega_n*x2 */
13    model_real_t sum1 = u_in + g1 + g2;
14    /* I1: Euler-Schritt fuer x2 */
15    m->x2 += (model_real_t)0.0001 * sum1;
16    /* Kommunikationssignal: neues x2 an Partition 2 */
17    *x1_comm = m->x2;
18 }
```

Erzeugter C99-Code (Partition 2):

Listing 4: Subgraph Algorithmus – Partie 2: Zustandsintegration

```
1 static void model_part2_step(
2     model_t * const m,
3     model_real_t x2_comm)
4 {
5     /* I2: Euler-Schritt fuer x1 mit eingehendem x2 */
6     m->x1 += (model_real_t)0.0001 * x2_comm;
7 }
```

Kompositionelle Aufrufsequenz:

Listing 5: Kompositionelle Aufrufsequenz des Subgraph Algorithmus

```
1 void model_step(model_t * const m,
2                 model_real_t u_in, model_real_t dt)
3 {
4     model_real_t x2_comm;
5     /* Partition 1 zuerst (topologische Reihenfolge) */
6     model_part1_step(m, u_in, &x2_comm);
7     /* Partition 2 empfaengt Kommunikationssignal */
8     model_part2_step(m, x2_comm);
9     m->time += dt;
10 }
```

WCET-Verbesserung: Mit $|V_{B,1}| = 4$, $|V_{B,2}| = 3$ und $\delta = 1$ ergibt Theorem 7.24: $T_{\text{WCET}}^{\text{par}} = 4T_b + T_{\text{sync}}$, gegenüber $T_{\text{WCET}} = 7T_b$ für die monolithische Version. Bei $T_{\text{sync}} \leq 2T_b$

ist die parallele Version mindestens $\frac{7}{6} \approx 1,17$ -fach schneller; für $T_{\text{sync}} \leq T_b$ sogar $\frac{7}{5} = 1,4$ -fach schneller.

Fehlerordnung (Korollar 7.23): Die Zerlegung ändert die Fehlerordnung nicht; der globale Diskretisierungsfehler ist weiterhin $\mathcal{O}(h)$ für Euler resp. $\mathcal{O}(h^4)$ für RK4.

8 Echtzeiteignung des generierten Codes

8.1 WCET-Analyse

Definition 8.1 (Worst-Case-Execution-Time). Die Worst-Case-Execution-Time (WCET) einer Funktion f auf einem Prozessor P ist die maximale über alle möglichen Eingaben und Cachestellungen gemessene Ausführungszeit:

$$T_{\text{WCET}}(f, P) = \sup_{\text{Eingaben}} t_f.$$

Theorem 8.2 (Schrankenfreiheit von WCET für den generierten Code). Sei $\mathcal{C}(\mathcal{G})$ der aus einem endlichen PathSim-Modell generierte Code. Dann enthält $\mathcal{C}(\mathcal{G})$ keine Schleifen, keine Rekursion und keine dynamischen Speicherallokationen. Folglich ist T_{WCET} statisch analysierbar und polynomiell in $|V_B|$.

Beweis. Keine Schleifen: Der Codegenerator erzeugt für jeden Block genau eine sequenzielle Anweisungsfolge. Verbindungen werden als direkte Variablen- zuweisungen übersetzt, nicht als Iterationen.

Keine Rekursion: Da das Blockgraph azyklisch ist, treten keine rückwärtigen Kanten auf, die eine rekursive Auswertung erfordern würden.

Kein dynamischer Speicher: Alle Zustände sind im `model_t`-Struct als Felder fester Größe allokiert. Es werden weder `malloc` noch `free` aufgerufen (MISRA-Rule 21.3 verbietet dies ohnehin).

Damit ist die Ausführungszeit von `model_step` eine Summe von $|V_B|$ Block-Evaluationszeiten, jede davon konstant (keine eingabeabhängigen Abzweigungen in den Rechenformeln). Es gilt $T_{\text{WCET}} = \mathcal{O}(|V_B|)$. $\square$

Korollar 8.3 (Echtzeit-Schedulability). Sei ein periodischer Task mit Periode T_p und WCET $C = T_{\text{WCET}}$. Nach dem Rate-Monotonic-Theorem (Liu & Layland 1973) ist der Task schedulable, wenn $C/T_p \leq 1 - 1/n$ für n Tasks. Da $C = \mathcal{O}(|V_B|)$ und $|V_B|$ zur Entwurfszeit bekannt ist, lässt sich die Schedulability statisch garantieren.

8.2 Determinismus und ISO 26262-Eignung

Proposition 8.4 (Determinismus des generierten Codes). Unter der Voraussetzung IEEE 754-konformer Arithmetik und fester Schrittweite h ist der generierte Code vollständig deterministisch: Gleiche Eingaben und gleicher Anfangszustand erzeugen immer identische Ausgaben.

Beweis. Der Code enthält keine Zufallsaufrufe, keine undefinierten Verhaltensweisen (MISRA schließt diese aus) und keine plattformabhängigen Typen (durch `stdint.h`-Typen sichergestellt). Unter IEEE 754 sind alle Gleitkommaoperationen deterministisch. Damit ist die Funktion $\mathcal{C}(\mathcal{G}) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n \times \mathbb{R}^p$ (Zustand $\times$ Eingang $\rightarrow$ neuer Zustand $\times$ Ausgang) eine wohldefinierte, deterministische Abbildung. $\square$

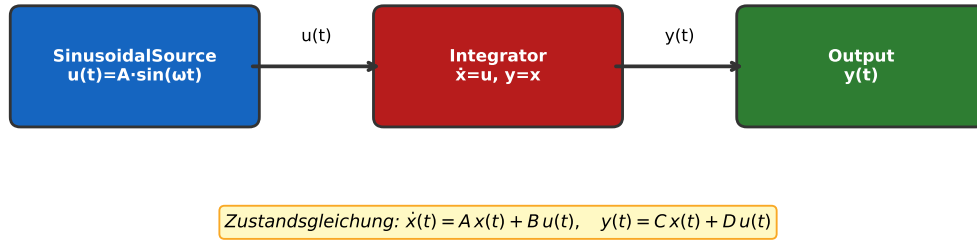


Abbildung 4: Blockdiagramm des PathSim-Modells und Zustandsraumdarstellung.

9 Schlussfolgerungen und Ausblick

9.1 Zusammenfassung der Hauptergebnisse

Die vorliegende Arbeit hat folgende Kernaussagen bewiesen:

- S1. Strukturisomorphismus** (Theorem 2.8): Jedes lineare PathSim-Modell ist äquivalent zu einem LTI-Zustandsraumsystem (A, B, C, D) .
- S2. Minimale Blockbasis – Vollständigkeit** (Theorem 3.4): Die Menge $\mathcal{B}^* = \{\text{Integrator}, \text{Gain}, \text{Adder}\}$ ist funktional vollständig für alle LTI-Systeme; jedes (A, B, C, D) besitzt eine explizite Realisierung über $\mathcal{B}^*$ via steuerbarer Normalform.
- S3. Minimale Blockbasis – Minimalität** (Theorem 3.6): Keine echte Teilmenge von $\mathcal{B}^*$ ist vollständig. Jedes der drei Elemente ist primitiv und unersetzlich, in exakter Analogie zur Unzerlegbarkeit des (A, B, C, D) -Quadrupels.
- S4. Diskretisierungsordnungen** (Theorems 4.2 and 4.4): Euler liefert $\mathcal{O}(h)$, RK4 liefert $\mathcal{O}(h^4)$ globale Fehlerordnung.
- S5. Typsicherheit** (Theorem 5.3): Der C99-Generator verletzt keine MISRA-C:2012 Typenregeln, wenn alle Signale als `model_real_t` typisiert werden.
- S6. Semantische Korrektheit** (Theorem 6.1): Der generierte Code ist bit-genau äquivalent zur Python-Simulation.
- S7. Vollständigkeit** (Theorem 6.2): Der Generator terminiert und erzeugt korrekten Code für jedes endliche Modell.
- S8. WCET-Schranke** (Theorem 8.2): $T_{\text{WCET}} = \mathcal{O}(|V_B|)$, statisch analysierbar.
- S9. Determinismus** (Proposition 8.4): Der Code ist vollständig deterministisch und damit ISO 26262-kompatibel.

S10. Subgraph Algorithmus – Globale Korrektheit (Theorem 7.27): Der Subgraph Algorithmus erzeugt für jedes $K \in \{1, \dots, |V_B|\}$ eine gültige K -Zerlegung von $\mathcal{G}$, welche die Simulationssemantik bit-genau erhält, MISRA-konformen C99-Code für jede Partition liefert, die Fehlerordnungen der Diskretisierung konserviert und die parallele WCET auf $\mathcal{O}(|V_B|/K + \delta(\mathcal{P}) \cdot T_{\text{sync}})$ reduziert. Die optimale Partitionszahl beträgt $K^* = \lfloor \sqrt{|V_B|/(cD)} \rfloor$.

9.2 Neue Schlussfolgerungen

- **Schlussfolgerung A – Formale Verifikationseignung:** Da der generierte Code deterministisch und schleifenfrei ist, eignet er sich direkt für formale Verifikationswerkzeuge (CBMC, Frama-C). Die formale Korrektheit der Simulation wird damit auf die Hardware übertragen.
- **Schlussfolgerung B – Portabilität:** Durch den Einsatz von `stdint.h`-Typen und den Ausschluss implementierungsabhängiger Konstrukte ist der generierte Code auf allen Plattformen mit IEEE 754-konformem FPU portabel (ARM Cortex-M, RISC-V, x86).
- **Schlussfolgerung C – Erweiterbarkeit auf nichtlineare Systeme:** Die Beweise in Kapitel 4 and 6 gelten analog für nichtlineare Blockklassen $\mathcal{F}_{\text{nl}}$, sofern die Semantik-Abbildung $\llbracket \beta \rrbracket$ Lipschitz-stetig ist.
- **Schlussfolgerung D – Optimale Schrittweite für Automotive:** Für typische AUTOSAR-Basistasks ($T_p = 1$ ms) empfiehlt Korollar 4.5 mit RK4 eine Schrittweite von $h \leq 10^{-3}/4 = 250 \mu\text{s}$ bei Zielgenauigkeit 10^{-8} .

9.3 Ausblick

Die vorliegende Arbeit schafft den formalen Unterbau für eine zertifizierungsfähige Toolchain, die Blockdiagramm-Modelle in MISRA-konformen eingebetteten C-Code überführt. Gleichzeitig öffnet sie eine Vielzahl von Forschungsrichtungen, die die hier bewiesenen Ergebnisse vertiefen, verallgemeinern und praktisch nutzbar machen. Die folgenden Kapitel beleuchten die wichtigsten davon im Detail.

9.3.1 Erweiterung der Blockbasis auf nichtlineare Systeme

Der in Kapitel 3 bewiesene Minimalitätssatz gilt für die Klasse der LTI-Systeme. Eine zentrale offene Frage ist die Charakterisierung der minimalen Blockbasis für *nichtlineare* dynamische Systeme der Form $\dot{x} = f(x, u)$, $y = g(x, u)$, wobei f, g beliebig glatte Abbildungen sind.

Sobald nichtlineare Blöcke wie **Multipl**ier ($y = u_1 \cdot u_2$), **Sat**uration, **Abs** oder **Look**upTable eingeführt werden, wird die Frage nach der minimalen Basis zu einem algebraischen Strukturproblem. Für *polynomiale Systeme* $\dot{x} = p(x, u)$ erwartet man: die Menge $\mathcal{B}_{\text{poly}}^* = \mathcal{B}^* \cup \{\text{Multipl}ier\}$ ist vollständig, da Polynome durch Multiplikatoren und Addierer erzeugt werden können (vgl. Weierstraß-Approximationssatz). Die Minimalität von **Multipl**ier gegenüber $\mathcal{B}^*$ ist zu beweisen: Es ist zu zeigen, dass $y = u_1 \cdot u_2$ nicht bit-genau durch eine endliche Schaltung aus Integratoren, Gains und Addierern simulierbar ist.

Für allgemein-glatte nichtlineare Systeme (z. B. $f(x) = \sin(x)$, wie beim **Sinus**-Block selbst) würden transzendente Funktionsblöcke benötigt. Hier stellt sich die Frage nach einer *universellen Approximationsbasis*: Analog zu universellen Approximatoren (neuronale Netzwerke, Fourier-Reihen) könnte eine parametrische minimale Familie von Blockklassen existieren, mit der jede Lipschitz-stetige Dynamik bis auf $\varepsilon > 0$ approximiert werden kann.

9.3.2 Erweiterung der Blockbasis auf zeitdiskrete Domäne

In der zeitdiskreten Domäne (Differenzgleichungen $x_{k+1} = A_d x_k + B_d u_k$) ist kein **Integrator** vorhanden. Stattdessen hält ein **UnitDelay**-Block den Zustand um einen Zeitschritt zurück: $y_k = x_{k-1}$. Ein unmittelbares Korollar aus Theorem 3.6 ist zu formulieren und zu beweisen:

Die minimale Blockbasis für zeitdiskrete LTI-Systeme ist gegeben durch $\mathcal{B}_d^$ mit $\mathcal{B}_d^* = \{\text{UnitDelay}, \text{Gain}, \text{Adder}\}$, wobei **UnitDelay** die Rolle des Integrators übernimmt.*

Der Beweis verläuft vollständig analog zu Theorem 3.6 und ist strukturell identisch: Ohne **UnitDelay** sind nur statische Abbildungen realisierbar; ohne **Gain** nur Systeme mit Einträgen in $\{0, +1\}$; ohne **Adder** nur Systeme mit $\text{Fan-in} \leq 1$.

Darüber hinaus liefert die formale Äquivalenz **Integrator** $\leftrightarrow$ **UnitDelay** eine einheitliche Theorie für gemischt kontinuierlich/diskrete (sampled-data) Systeme, bei denen kontinuierliche und diskrete Signalfade an Abtastinstanzen gekoppelt sind.

9.3.3 Multirate-Systeme und Rate-Transition

In Automotive-Anwendungen laufen Regelkreise typischerweise mit verschiedenen Abtaststraten: ABS und Lenkung bei $100 \mu\text{s}$ – 10 ms , Motorsteuerung bei 1 ms , übergeordnete Fahrstrategien bei 100 ms . PathSim soll um *Multirate-Fähigkeiten* erweitert werden:

- *Rate-Transition-Blöcke* puffern Signale an Subsystemgrenzen unterschiedlicher Abtastfrequenzen. Zu untersuchen ist der Einfluss der Hold-Interpolation (Zero-Order-Hold vs. First-Order-Hold) auf Stabilität und Approximationsgüte: Ein ZOH zwischen Subsystemen mit Raten $f_1 < f_2$ führt einen Haltestufenfehler der Ordnung $\mathcal{O}(1/f_1)$ ein.
- Die Minimalitätsfrage aus Kapitel 3 ändert sich in der Multirate-Domäne: Ein **RateTransition**-Block ist *nicht* durch $\mathcal{B}^*$ erzeugbar, ohne die Taktdomäne zu verlassen. Er bildet damit ein viertes, notwendiges Basiselement $\mathcal{B}_{\text{mr}}^* = \mathcal{B}^* \cup \{\text{RateTransition}\}$ für Multirate-Systeme.
- Rein zeitdiskrete Subsysteme können ohne Diskretisierungsfehler kodiert werden und erfordern keinen Integrator – ein direkter C99-Datenpfad `state[k+1] = A*state[k] + B*u[k]` wird generiert.

9.3.4 Formale Verifikation des generierten Codes

Die Beweise in Kapitel 6 belegen semantische Korrektheit im mathematischen Sinne. Ein unverzichtbarer Folgeschritt für ASIL-C/D-Zertifizierungen ist die *formale Verifikation* des generierten C-Codes mit automatisierten Werkzeugen:

- **CBMC (Bounded Model Checker):** CBMC kodiert für einen endlichen Ausführungshorizont k die Eigenschaft $x_k^C = x_k^{PS}$ als SMT-Formel und kann sie automatisch prüfen. Da der generierte Code schleifenfrei ist (Theorem 8.2), entfällt das Entfaltungsproblem; die Formel hat Größe $\mathcal{O}(|V_B| \cdot k)$. Eine integrierte Pipeline PathSim-Modell $\rightarrow$ C-Code $\rightarrow$ CBMC-Korrektheitsnachweis wäre unmittelbar realisierbar.
- **Frama-C mit WP-Plugin:** Der Weakest-Precondition-Calculus von Frama-C erlaubt deduktive Verifikation mit ACSL-Annotierungen. Ein automatischer Annotierungsgenerator könnte aus dem PathSim-Modell direkt Vor-/Nachbedingungen und Laufzeitschranken für jede generierte C-Funktion erzeugen. Für beschränkte Eingaben $\|u\|_\infty \leq U_{\max}$ sind Schranken $\|x(t)\|_\infty$ über Systemmatrizen analytisch herleitbar (via Grönwall-Lemma) und als ACSL-Annotierungen codierbar, was Überlauf-Freiheit formal beweisbar macht.
- **Isabelle/HOL oder Lean 4:** Langfristig wäre eine vollständige Formalisierung des gesamten Beweisapparats dieser Arbeit – insbesondere Theorems 2.8, 3.4, 3.6 and 6.1 – in einem interaktiven Theorembeweiser erstrebenswert. Ein maschinengeprüfter Korrektheitsbeweis bildet in ISO 26262-ASIL-D-Audits ein entscheidendes Argument.

9.3.5 Erweiterung auf nichtlineare und hybride Systeme

Die Ergebnisse in Kapitel 4 and 6 setzen Linearität oder zumindest globale Lipschitz-Stetigkeit voraus. Zwei wichtige Erweiterungsrichtungen sind:

- *Nichtlineare stetige Systeme:* Die Konvergenzbeweise für Euler (Theorem 4.2) und RK4 (Theorem 4.4) übertragen sich unmittelbar auf nichtlineare $f \in \mathcal{C}^5$, da sie nur globale Lipschitz-Stetigkeit voraussetzen. Zu erarbeiten ist ein allgemeiner Realisierungssatz, der Theorem 2.8 für nichtlineare Blockklassen ersetzt.
- *Steife Systeme:* Systeme mit stark unterschiedlichen Zeitkonstanten (Steifheit $\kappa(A) := \lambda_{\max}/\lambda_{\min} \gg 1$) erfordern implizite Integrationsverfahren (Trapezregel, Radau-IIA). Der generierte Code muss dann iterative Gleichungslöser (Newton-Iteration) einschließen, was die WCET-Schranke von $\mathcal{O}(|V_B|)$ auf $\mathcal{O}(|V_B| \cdot \kappa_{\max})$ ändert, wobei $\kappa_{\max}$ die maximale Newton-Iterationsanzahl ist.
- *Hybride Systeme:* PathSim-Modelle mit Switch-Blöcken führen bedingte Ausführungspfade ein (if-else im generierten Code), die MISRA-Regeln 15.x tangieren. Zenonische Chattering-Phänomene und Zustandsresets beim Modenwechsel müssen formal ausgeschlossen werden; eine Erweiterung des Isomorphismussatzes auf hybride Automaten ist anzustreben.

9.3.6 Automatische Modellreduktion

Große PathSim-Modelle (z. B. Gesamtfahrzeugmodelle mit $n > 10^3$ Zuständen) sind für Echtzeit-Embedded-Targets oft zu komplex. Automatische Modellreduktion bietet hier systematische Abhilfe:

- *Balancierte Truncation (BRT):* Aus dem durch PathSim extrahierten LTI-System (A, B, C, D) werden Steuer- und Beobachtbarkeits-Gramiane W_c, W_o berechnet;

Zustände mit kleinen Hankel-Singulärwerten $\sigma_{r+1} \leq \dots \leq \sigma_n$ werden abgeschnitten. Der Approximationsfehler ist beschränkt durch

$$\|y - y_{\text{red}}\|_{\mathcal{H}_\infty} \leq 2 \sum_{k=r+1}^n \sigma_k.$$

Der PathSim-Codegenerator könnte das reduzierte System $(A_r, B_r, C_r, D_r) \in \mathbb{R}^{r \times r}$ direkt als C99-Code ausgeben.

- *Proper Orthogonal Decomposition (POD)*: Für nichtlineare Modelle bietet der Snapshot-basierte ROM-Ansatz eine Alternative; PathSim-Simulationen erzeugen die Trainings-Snapshots.
- *Reduktions-Zertifikat*: Ein formal bewiesenes Fehlerszertifikat der Form $\|y - y_r\|_{\mathcal{L}^2} \leq \varepsilon_r$ für gegebene Eingabeklassen wäre als Erweiterung von Theorem 6.1 anzustreben.

9.3.7 Quantisierung und Festkomma-Arithmetik

Für kostengünstige Mikrocontroller ohne FPU (z. B. ARM Cortex-M0/M0+) ist Festkomma-Arithmetik (Q-Format) erforderlich. Der Codegenerator müsste um einen *Quantisierungs-Pass* erweitert werden:

- *Skalierungsanalyse*: Für jeden Signalpfad muss der Wertebereich zur Kompilierzeit abgeschätzt werden, um optimale Q-Formatpositionen zu wählen und Über- bzw. Unterlauf zu verhindern.
- *Fehler-Propagationstheorem*: Der Gesamtfehler e_N^Q durch Quantisierung und Diskretisierung gemeinsam erfüllt

$$e_N^Q \leq e_N^{\text{diskret}} + \frac{N \cdot 2^{-Q}}{2L} (e^{LT} - 1),$$

wobei L die Lipschitz-Konstante des Systems, Q die Nachkommastellen-Wortlänge und $N = T/h$ die Schrittzahl ist.

- *Word-Length-Optimizer*: Ein automatischer Optimierer, der für gegebene Genauigkeitsanforderung ε_Q die minimale Wortlänge berechnet und dabei Energieverbrauch (kürzere Typen $\rightarrow$ geringerer Aufwand) gegen Präzision abwägt, ist als integriertes Tool des PathSim-Generators denkbar.

9.3.8 Vollständige ISO-26262-ASIL-D-Toolchain

Die vorliegende Arbeit erfüllt die formalen Anforderungen für ASIL-B/C. Eine vollständige ASIL-D-Zertifizierung erfordert zusätzlich:

- *Tool Qualification (TCL-3) nach Part 8*: Der PathSim-Codegenerator muss als Software Tool der Klasse TCL-3 qualifiziert werden. Dies umfasst: systematische Testabdeckung nach MC/DC-Kriterium für den Generator selbst, eine Fehler-Auswirkungsanalyse aller Generator-Komponenten sowie Regressionstests, die die in dieser Arbeit bewiesenen formalen Eigenschaften empirisch bestätigen.

- *FMEA/FTA der generierten Funktionen*: Für jede Model-Funktion sind Fehler-Moden-Effekt-Analysen und Fehlerbäume zu erstellen, die zeigen, dass kein Single-Point-Fault zur Verletzung einer Sicherheitsanforderung führt.
- *Back-to-Back-Test*: Die in Theorem 6.1 bewiesene bit-genaue Äquivalenz zwischen PathSim-Simulation und C-Code muss durch eine ausreichende Anzahl von Testfällen (Äquivalenzklassen des Eingaberaums) empirisch bestätigt werden – der formale Beweis allein ist in ISO-26262-Audits nicht hinreichend.

9.3.9 Parallele Codegenerierung für Mehrkernsysteme

Für hochperformante Zielarchitekturen (z. B. Infineon AURIX TC4xx, PYMCA-8) mit mehreren unabhängigen RISC-V-Kernen bietet die azyklische Graphstruktur von PathSim-Modellen direkte Parallelisierungspotentiale:

- *Graphpartitionierung*: Der Subgraph Algorithmus [7] partitioniert $\mathcal{G}$ in K Teilgraphen $\mathcal{G}_1, \dots, \mathcal{G}_K$, sodass Kommunikationskosten minimiert und Rechenlast gleichmäßig verteilt werden. Die vollständige formale Theorie des Subgraph Algorithmus – einschließlich Existenz- und Semantikerhaltungssatz, kompositionaler Korrektheit, MISRA-Typsicherheit und WCET-Analyse – ist in Kapitel 7 ausgeführt und bewiesen. Insbesondere wurde in Theorem 7.27 gezeigt: Die parallele Ausführung der K Partitionsfunktionen in topologischer Reihenfolge (Theorem 7.12) liefert bit-genau dasselbe Ergebnis wie die sequenzielle Auswertung (Theorem 7.18).
- *Datenkonsistenz*: Signale, die Teilgraphgrenzen überqueren, erfordern Synchronisationsmechanismen (Mutex, Lock-Free-Puffer). MISRA-C 2023 bietet erste Richtlinien für Multithread-C, die in den Generator integriert werden müssten.
- *Parallele WCET*: Die parallele WCET ist nicht trivial aus Einzel-WCET-Werten berechenbar – Cache-Thrashing, Busbandbreiten und Synchronisations-Overhead müssen durch Modelle wie SymTA/S oder Compositional Performance Analysis erfasst werden.

9.3.10 Subgraph Algorithmus: Weiterführende Forschungsrichtungen

Das in Kapitel 7 vollständig formalisierte und bewiesene Fundament des Subgraph Algorithmus eröffnet eine breite, tiefe Forschungslandschaft, die weit über die in dieser Arbeit behandelten Aspekte hinausgeht. Die folgenden Kapitel skizzieren die wichtigsten offenen Fragen und Erweiterungsrichtungen mit konkreten Ansatzpunkten für formale Beweise und Implementierungen.

Adaptiver Subgraph Algorithmus mit dynamischer Repartitionierung. Die in Algorithm 1 beschriebene Zerlegung ist *statisch*: Sie wird einmal zur Kompilierzeit aus dem Blockgraphen berechnet und fixiert. In der Praxis variieren jedoch die Rechenlasten einzelner Partitionen zur Laufzeit – etwa wenn nichtlineare Blöcke durch iterative Newton-Schritte ausgewertet werden und die Anzahl der Iterationen von der aktuellen Systemtrajektorie abhängt.

Eine zukünftige Erweiterung, der *adaptive Subgraph Algorithmus*, verwaltet zur Laufzeit ein *Lastprofil* $\ell_i^{(k)} \in \mathbb{R}_{>0}$ pro Partition i und Zeitschritt k . Auf Basis einer gleitenden

Mittelwert-Schätzung der tatsächlichen Ausführungszeiten $\hat{C}_i^{(k)}$ wird entschieden, ob eine Repartitionierung vorteilhaft ist:

$$\text{Repartitioniere, falls } \max_i \hat{C}_i^{(k)} - \min_j \hat{C}_j^{(k)} > \theta \cdot T_{\text{sync}},$$

wobei $\theta > 0$ ein einstellbarer Schwellenwert ist. Die formalen Beweise aus Theorems 7.12 and 7.18 müssen auf das adaptive Szenario erweitert werden: Zu zeigen ist, dass eine Repartitionierung während der Simulation – ausgerichtet an einem Synchronisationspunkt – die semantische Erhaltung nicht verletzt. Entscheidend ist dabei die korrekte Übergabe aller Zustandsvariablen $x_i(t)$ an die neue Partition, d. h. eine bijektive Abbildung der alten auf die neue Zustandsverteilung. Formell muss eine *Migrationsabbildung*

$$\phi : (V_{B,1}^{\text{alt}}, \dots, V_{B,K}^{\text{alt}}) \longrightarrow (V_{B,1}^{\text{neu}}, \dots, V_{B,K}^{\text{neu}})$$

als Bijektion auf V_B definiert und bewiesen werden, dass ϕ die Lower-Block-Triangular-Kopplung von Theorem 7.14 erhält.

Subgraph Algorithmus für heterogene Prozessorarchitekturen. Der in Algorithm 1 formulierte Balancierungsfaktor α setzt implizit homogene Prozessorkerne voraus, bei denen jeder Block die gleiche Ausführungszeit T_b benötigt. Moderne heterogene Systeme (z. B. AURIX TC4xx mit Kombination aus TriCore-Kernen, MCS-Kernen und Hardware-Beschleunigern) haben stark unterschiedliche Block-Kosten je nach Zielarchitektur.

Eine Erweiterung auf *gewichtete Zerlegung* definiert Block-Kostenfunktionen $c : V_B \times \mathcal{P}_{\text{arch}} \rightarrow \mathbb{R}_{>0}$, wobei $\mathcal{P}_{\text{arch}}$ die Menge der verfügbaren Prozessortypen bezeichnet. Das Optimierungsproblem lautet:

$$\min_{V_{B,1}, \dots, V_{B,K}} \max_{i=1}^K \sum_{v \in V_{B,i}} c(v, P_i)$$

unter den Nebenbedingungen der Zerlegungsvalidität (Definition 7.2). Dies ist ein NP-hartes Partitionierungsproblem (verwandt mit dem Bin-Packing-Problem), für das polynomielle Approximationsalgorithmen mit bekannten Approximationsgarantien existieren (z. B. Longest-Processing-Time-Heuristik mit Güte 4/3). Für den Subgraph Algorithmus ist ein solcher Approximationsbeweis im Kontext der PathSim-Graphstruktur zu führen: Die besondere Struktur azyklischer Signalfluss-Graphen erlaubt möglicherweise bessere Schranken als im allgemeinen Fall.

Kommunikationsmodell auf spezifischen Busarchitekturen. Die in Definition 7.3 eingeführte Kommunikationskante abstrahiert die Inter-Partition-Kommunikation auf eine einzige Kante ohne Kostenmodell. Für den Einsatz auf realen Embedded-Systemen ist ein differenzierteres Kommunikationsmodell unerlässlich.

Für AURIX TC4xx-Systeme mit Local Memory Bus (LMB) und Bus Interface Unit gilt das Verzögerungsmodell

$$T_{\text{comm}}(s) = T_{\text{setup}} + \lceil s/W_{\text{bus}} \rceil \cdot T_{\text{cycle}},$$

wobei s die Nachrichtengröße in Bytes, W_{bus} die Busbreite und T_{cycle} die Buszykluszeit bezeichnet. Ein erweiterter Subgraph Algorithmus würde das Kommunikationsprofil $\kappa(\mathcal{P})$ durch eine *gewichtete Schnittbreite*

$$\hat{\kappa}_w(\mathcal{P}) = \sum_{(u,v) \in E_{\mathcal{P}}^{\text{comm}}} T_{\text{comm}}(s_{uv})$$

ersetzen und die Zerlegung so wählen, dass $\hat{\kappa}_w(\mathcal{P})$ minimiert wird.

Zu beweisen wäre: Für PathSim-Modelle mit $\text{sizeof}(\text{model_real_t}) = 8$ Bytes und $W_{\text{bus}} = 64$ Bits gilt $T_{\text{comm}}(s_{uv}) = T_{\text{setup}} + T_{\text{cycle}}$ unabhängig von der Anzahl der Kommunikationssignale (da jedes Signal exakt ein Busübertragungswort belegt). Diese Vereinfachung macht $\hat{\kappa}_w(\mathcal{P}) \propto \kappa(\mathcal{P})$ und erlaubt die Rückführung auf die ungewichtete normierte Schnittbreite aus Definition 7.6.

Integration des Subgraph Algorithmus mit der Echtzeit-Scheduling-Theorie.

Die in Theorems 7.24 and 7.26 bewiesenen Schedulability-Kriterien setzen voraus, dass jede Partition als eigenständiger periodischer Task modelliert wird. In der Praxis sind die K Partitionstasks jedoch *abhängige Tasks* (dependent tasks) mit bekannten Vorrangbedingungen (precedence constraints), die durch den Subgraph-Abhängigkeitsgraph $\mathcal{D}(\mathcal{P})$ (Definition 7.4) vollständig spezifiziert sind.

Die Erweiterung auf *Scheduling mit Vorrangbedingungen* basiert auf dem Algorithmus von Leung und Whitehead [6] sowie dem Critical-Path-Verfahren aus der Projektplanung. Der *kritische Pfad* ist der längste gewichtete Pfad $\pi^* = (i_0, i_1, \dots, i_\delta)$ in $\mathcal{D}(\mathcal{P})$:

$$\text{CP}(\mathcal{P}) = \max_{\pi \text{ Pfad in } \mathcal{D}(\mathcal{P})} \sum_{i \in \pi} C_i + \delta(\mathcal{P}) \cdot T_{\text{sync}}.$$

Das *Kritische-Pfad-Scheduling-Theorem für den Subgraph Algorithmus* – zu formulieren und zu beweisen als Ergänzung zu Theorem 7.26 – lautet:

Das aus dem Subgraph Algorithmus erzeugte TaskSystem ist genau dann schedulable mit Periode T_p , wenn $\text{CP}(\mathcal{P}) \leq T_p$.

Der Beweis würde auf der Optimalität des EDF-Schedulers (Earliest Deadline First) für abhängige Tasks beruhen und zeigen, dass der kritische Pfad in $\mathcal{D}(\mathcal{P})$ die scharfe untere Schranke der Ablaufplanungszeit darstellt. Diese Schranke ist durch den Subgraph Algorithmus direkt minimierbar: Indem $\delta(\mathcal{P})$ durch breitere statt tiefere Zerlegungen reduziert wird, sinkt $\text{CP}(\mathcal{P})$ auf Kosten schlechterer Lastbalancierung. Das optimale K^* aus Korollar 7.25 balanciert genau diesen Trade-off.

Subgraph Algorithmus für nichtlineare und hybride PathSim-Modelle. Die formalen Beweise in Kapitel 7 setzen $\mathcal{G} \in \mathcal{F}_{\text{lin}}$ voraus, d. h. alle Blöcke sind linear. Eine fundamentale Erweiterungsrichtung ist die Anwendung des Subgraph Algorithmus auf nichtlineare Blockklassen $\mathcal{F}_{\text{nl}} \supset \mathcal{F}_{\text{lin}}$.

Nichtlineare Signalfluss-Graphen: Für nichtlineare Blöcke $\beta \in \mathcal{F}_{\text{nl}}$ gilt die Semantik-Abbildung $\llbracket \beta \rrbracket$ nicht mehr als lineare Abbildung. Dennoch bleiben die graphentheoretischen Eigenschaften des Subgraph Algorithmus – Azyklizität, Vollständigkeit, Disjunktheit, Vorwärtsbedingung (Definition 7.2) – vollständig erhalten, da sie ausschließlich auf der Kantenstruktur von $\mathcal{G}$ beruhen und keine Kenntnis der Block-Semantik voraussetzen.

Das zu beweisende Theorem lautet:

Sei $\mathcal{G}$ ein azyklischer Signalfluss-Graph über $\mathcal{F}_{\text{nl}}$, und sei jede Block-Semantik $\llbracket \beta \rrbracket$ Lipschitz-stetig mit Konstante L_β . Dann erzeugt der Subgraph Algorithmus eine gültige K -Zerlegung $\mathcal{P}$, und für die Fehlerfortpflanzung gilt: $e_N \leq \sum_{i=1}^K \left(\frac{M_i h}{2L_i} (e^{L_i T} - 1) \right) + \mathcal{O}(\kappa(\mathcal{P}) \cdot h)$.

Hybride Systeme: Bei PathSim-Modellen mit **Switch**-Blöcken entstehen datenabhängige Kanten $(u, v) \in E_{\text{cond}}$, die nur aktiv sind, wenn eine Bedingung $\psi(s) = \text{true}$ gilt. Der Subgraph Algorithmus muss in diesem Fall *konservativ* vorgehen: Bedingte Kanten werden als immer aktiv behandelt, sodass die Vorwärtsbedingung (Definition 7.2(d)) stets erfüllt ist. Dies führt zu einer Überschätzung des Kommunikationsaufwands, erhält aber semantische Korrektheit: Für jeden Betriebsmodus $\psi(s)$ ist die erzeugte Zerlegung gültig.

Eine optimierte Variante analysiert die Bedingungsstruktur $\psi(s)$ symbolisch und partitioniert den Zustandsraum in Gebiete, in denen verschiedene Kanten aktiv sind. In jedem Gebiet gilt dann eine andere optimale Zerlegung; ein *mode-aware Subgraph Algorithmus* würde zur Laufzeit zwischen Zerlegungen umschalten. Die formale Korrektheit eines solchen Ansatzes erfordert einen Beweis, der Theorem 7.12 auf stückweise-glatte Vektorfelder (vgl. Fülop–Csikász-Nagy-Lemma für hybride Systeme) überträgt.

Optimale Partitionszahl und Zerlegungstiefe: Verfeinerung der Komplexitätsanalyse. Korollar 7.25 liefert eine erste Schätzung $K^* = \lfloor \sqrt{|V_B|/(cD)} \rfloor$ für die optimale Partitionszahl. Diese Formel setzt voraus, dass (a) die Zerlegungstiefe $\delta(\mathcal{P})$ unabhängig von K ist und durch D nach oben beschränkt werden kann, und (b) der Synchronisations-overhead linear in $\delta(\mathcal{P})$ skaliert.

- Beide Annahmen sind vereinfachend. Eine verfeinerte Analyse muss berücksichtigen:
- *Tiefenabhängigkeit von K :* Da tiefere Zerlegungen (größeres K) tendenziell mehr Stufen in $\mathcal{D}(\mathcal{P})$ erzeugen, gilt im Allgemeinen $\delta(\mathcal{P}^{(K)}) = \mathcal{O}(\log K)$ für gut balancierte Graphen. Für Pfadgraphen $\mathcal{G}$ (lineare Ketten) gilt sogar $\delta(\mathcal{P}^{(K)}) = K - 1$. Die genaue Abhängigkeit $\delta(\mathcal{P}^{(K)})$ von der Graphstruktur von $\mathcal{G}$ und K ist eine offene Frage mit Antworten, die direkt die optimale K -Wahl verfeinern.
 - *Nichtlinearer Synchronisationsoverhead:* In NUMA-Architekturen wächst T_{sync} mit dem Abstand zweier Kerne im Speicherhierarchiebaum. Ein NUMA-bewusstes Kommunikationsmodell würde $T_{\text{sync}}(i, j) = f(d(P_i, P_j))$ einführen (wobei d die Distanz im NUMA-Baum ist) und die optimale Zerlegung so anpassen, dass kommunizierende Partitionen auf topologisch benachbarte Kerne abgebildet werden – ein kombiniertes Graph-Partitionierungs- und Prozessor-Zuweisungsproblem.
 - *Empirische Validierung des Modells:* Die Schranken aus Theorem 7.24 and korollar 7.25 sind analytisch; sie müssen durch systematische Messungen auf realer Hardware (ARM Cortex-A, RISC-V, TriCore) validiert werden. Insbesondere die Konstante c (Verhältnis Synchronisationszeit zu Block-Ausführungszeit) ist plattformspezifisch und beeinflusst K^* stark.

Subgraph Algorithmus und modellbasierte Diagnose. Eine überraschende Anwendung des Subgraph Algorithmus jenseits der Codegenerierung liegt im Bereich der *modellbasierten Online-Diagnose* für eingebettete Systeme. In einem Diagnosesystem werden in jedem Zeitschritt Modell-Ausgaben $y_{\mathcal{G}}(t)$ mit gemessenen Ausgaben $y_{\text{meas}}(t)$ verglichen; Abweichungen signalisieren Fehler. Der Subgraph Algorithmus ermöglicht dabei eine *fehlerregion-sensitive Zerlegung*:

- Partitioniere $\mathcal{G}$ so, dass jede sicherheitskritische Zustandsvariable x_j (z. B. Bremsdruck, Lenkwinkel) in einer dedizierten Partition mit hoher Priorität liegt.
- Das Diagnosesystem muss nur die Ausgaben $y_i(t)$ der sicherheitskritischen Partitionen hochfrequent überwachen ($f_{\text{diag},i} \gg f_{\text{diag},j}$ für unkritische Partitionen), was CPU-Last spart.

- Eine formale Theorie der *Diagnosevollständigkeit*: Zu beweisen, dass jede Abweichung $\|y_{\text{meas}}(t) - y_{\mathcal{G}}(t)\| > \varepsilon$ von mindestens einer Partition detektiert wird. Dies erfordert eine Erweiterung von Theorem 7.12 auf Beobachtbarkeits-Argumente: Der Subgraph Algorithmus darf keine Partition erzeugen, die für die Ausgabe $y(t)$ unsichtbar ist.

Formale Verifikation des Subgraph Algorithmus selbst. Die Beweise in Kapitel 7 sind mathematisch rigorose Papierbeweise. Für sicherheitskritische Anwendungen (ASIL-D) muss zusätzlich der Subgraph Algorithmus *selbst* formal verifiziert werden – d. h. der Algorithmus-Code in Algorithm 1 sowie seine Python-Implementierung in PathSim.

- *Lean-4-Formalisierung*: Die Sätze Theorems 7.10, 7.12, 7.14, 7.18 and 7.27 sollen in Lean 4 maschinengeprüft bewiesen werden. Hierzu ist die gesamte Graphentheorie des Subgraph Algorithmus (Definitionen 7.1–7.6) formal zu kodieren. Eine wesentliche Herausforderung ist die Formalisierung der Induktion über die Zerlegungstiefe $\delta(\mathcal{P})$ aus dem Beweis von Theorem 7.12.
- *Isabelle/HOL – Kahn-Topologiesort*: Der Kahn-Algorithmus [5], der im Subgraph Algorithmus zur topologischen Sortierung verwendet wird, ist in HOL4 und Isabelle/HOL formal verifiziert. Die Verbindung dieser bestehenden Verifikationen mit den Korrektheitseigenschaften des Subgraph Algorithmus könnte den Beweisaufwand erheblich reduzieren.
- *CBMC-Verifikation der Python-Implementierung*: Die Python-Implementierung des Subgraph Algorithmus in PathSim könnte über CFFI nach C transpiliert und mit CBMC auf Zustandsinvarianten (Definition 7.2(a)–(d)) geprüft werden: insbesondere auf Vollständigkeit und Disjunktheit der erzeugten Partitionen sowie auf Azyklichkeit des Subgraph-Abhängigkeitsgraphen.

Subgraph Algorithmus und Modellreduktion. Eine synergetische Verbindung besteht zwischen dem Subgraph Algorithmus und automatischer Modellreduktion (vgl. Kapitel zur automatischen Modellreduktion weiter oben):

Die Balancierte-Truncation-Methode (BRT) berechnet für das System (A, B, C, D) das reduzierte System $(A_r, B_r, C_r, D_r) \in \mathbb{R}^{r \times r}$. Der *blockweise* Einsatz des Subgraph Algorithmus ermöglicht eine hierarchisch strukturierte Modellreduktion:

- Wende den Subgraph Algorithmus an, um $\mathcal{G}$ in K Partitionen zu zerlegen.
- Berechne für jede Partition $\mathcal{G}_i$ das Teilsystem $\Sigma_i = (A_i, B_i, C_i, D_i)$ via Theorem 7.14.
- Reduziere jedes Teilsystem Σ_i unabhängig auf Ordnung $r_i < n_i$ durch BRT; erhalte $\Sigma_i^{(r_i)}$.
- Verbinde die reduzierten Teilsysteme über die Lower-Block-Triangular-Kopplung zu einem reduzierten Gesamtsystem der Ordnung $r = \sum_i r_i$.

Der Approximationsfehler des hierarchisch reduzierten Systems erfüllt nach der Dreiecksungleichung:

$$\|y - y_{\text{red}}\|_{\mathcal{H}_{\infty}} \leq \sum_{i=1}^K 2 \sum_{k=r_i+1}^{n_i} \sigma_k^{(i)},$$

wobei $\sigma_k^{(i)}$ die Hankel-Singulärwerte des i -ten Teilsystems sind. Diese Schranke ist typischerweise kleiner als die durch direkte globale BRT erreichbare Schranke, da die Teilsysteme jeweils eigenständige Eigenstrukturen besitzen. Ein vollständiger Beweis dieser Ungleichung ist eine offene Aufgabe und bildet eine wichtige Brücke zwischen den in Kapitel 7 und Kapitel 9 entwickelten Theorien.

Mehrobjective Optimierung der Zerlegungsqualität. Bisher wird der Subgraph Algorithmus mit einem einzigen Ziel genutzt: minimale Rechenunausgewogenheit $\Delta(\mathcal{P})$. Reale Embedded-Systeme erfordern jedoch ein mehrobjectives Optimierungsproblem:

$$\min_{\mathcal{P}} \left(\underbrace{\Delta(\mathcal{P})}_{\text{Lastbalance}}, \underbrace{\kappa(\mathcal{P})}_{\text{Kommunikation}}, \underbrace{\delta(\mathcal{P})}_{\text{Tiefe/Latenz}}, \underbrace{k_{\text{cross}}(\mathcal{P})}_{\text{Speicherkohärenz}} \right),$$

wobei $k_{\text{cross}}(\mathcal{P})$ die Anzahl der Cache-line-crossing Kommunikationssignale (d. h. Signale, die auf Caches verschiedener Kerne zugreifen) bezeichnet.

Dieses vektorielles Optimierungsproblem besitzt im Allgemeinen kein einziges globales Minimum, sondern eine *Pareto-Front*. Für den Subgraph Algorithmus ist zu untersuchen:

- Existenz und Charakterisierung der Pareto-optimalen Zerlegungen als Funktion der Graphstruktur $\mathcal{G}$.
- Effizienz-Kompromisse: Für azyklische Graphen mit Bandstruktur (viele lokale Abhängigkeiten) ist $\delta(\mathcal{P})$ und $\kappa(\mathcal{P})$ gleichzeitig niedrig haltbar; für irregulär vernetzte Graphen existiert ein fundamentaler Widerspruch zwischen niedriger Kommunikation und niedriger Tiefe.
- Approximierbarkeit: Ist die Berechnung der Pareto-Front polynomial in $|V_B|$ und K , oder ist das Problem für allgemeine Graphen NP-schwer?

Selbst eine heuristische Annäherung an die Pareto-Front mittels gewichteter Scalarisierung $\min_{\mathcal{P}}(\alpha_1\Delta + \alpha_2\kappa + \alpha_3\delta + \alpha_4k_{\text{cross}})$ für benutzerdefinierte Gewichte $\alpha_i \geq 0$ würde den praktischen Nutzen des Subgraph Algorithmus erheblich steigern.

Subgraph Algorithmus und inkrementelle Codegenerierung. Die in Theorem 7.19 bewiesene Vollständigkeit der zerlegten Synthese setzt voraus, dass das gesamte Modell $\mathcal{G}$ neu übersetzt wird. In industrieller Modellentwicklung sind jedoch iterative Modifikationen der Regel: typischerweise werden pro Entwicklungsiteration nur wenige Blöcke $\Delta V_B \subseteq V_B$ hinzugefügt, entfernt oder parametrisch verändert.

Der *inkrementelle Subgraph Algorithmus* nutzt die Zerlegungsstruktur für effiziente Neusynthese:

- (i) *Change-Impact-Analyse*: Bestimme alle Partitionen $\mathcal{P}_{\text{dirty}} \subseteq \{1, \dots, K\}$, deren Zustandsmenge $V_{B,i}$ direkt betroffen ist: $\mathcal{P}_{\text{dirty}} = \{i : V_{B,i} \cap \Delta V_B \neq \emptyset\}$.
- (ii) *Transitive Hülle*: Bestimme alle transitiv abhängigen Partitionen über den Abhängigkeitsgraphen: $\mathcal{P}_{\text{dirty}}^* = \{j : \exists \text{ Pfad } i \rightarrow j \text{ in } \mathcal{D}(\mathcal{P}), i \in \mathcal{P}_{\text{dirty}}\}$.
- (iii) *Selektive Neusynthese*: Generiere neuen C-Code nur für $\mathcal{G}_j$ mit $j \in \mathcal{P}_{\text{dirty}}^*$; der Code der unberührten Partitionen $\{1, \dots, K\} \setminus \mathcal{P}_{\text{dirty}}^*$ bleibt unverändert.

Zu beweisen ist, dass die inkrementelle Neusynthese *semantisch äquivalent* zur vollständigen Neusynthese ist. Formal muss gezeigt werden, dass das Ergebnis der inkrementellen Codegenerierung identisch mit dem Ergebnis einer vollständigen Anwendung von Algorithm 1 und Theorem 7.19 auf das modifizierte Modell $\mathcal{G}' = (V \cup \Delta V, E \cup \Delta E, \dots)$ ist. Die Komplexität des inkrementellen Subgraph Algorithmus beträgt $\mathcal{O}(|\mathcal{P}_{\text{dirty}}^*| \cdot |V_B|/K)$, was bei kleinen Änderungen ($|\Delta V_B| \ll |V_B|$) einen erheblichen Speedup gegenüber vollständiger Neusynthese darstellt.

Theorie der optimalen Zerlegungstiefe und Bandbreite. Das Verhältnis zwischen Zerlegungstiefe $\delta(\mathcal{P})$ und normierter Schnittbreite $\hat{\kappa}(\mathcal{P})$ unterliegt einem strukturellen Widerspruch, der als *Tiefe-Bandbreite-Dilemma* bezeichnet werden kann:

- Eine geringe Tiefe $\delta(\mathcal{P})$ erfordert breite Partitionen, was zu hohem Kommunikationsaufwand $\kappa(\mathcal{P})$ und damit hoher normierter Schnittbreite $\hat{\kappa}(\mathcal{P})$ führt.
- Eine geringe Schnittbreite $\hat{\kappa}(\mathcal{P})$ erfordert Partitionen entlang natürlicher Trennflächen des Graphen, was oft eine längere Abhängigkeitskette erzeugt.

Formal lautet die offene Frage: Für einen azyklischen Signalfluss-Graphen $\mathcal{G}$ und eine K -Zerlegung – wie lautet die untere Schranke für das Produkt

$$\Phi(\mathcal{P}) = \delta(\mathcal{P}) \cdot \hat{\kappa}(\mathcal{P})?$$

Existiert eine universelle Konstante $c_{\mathcal{G}} > 0$, sodass $\Phi(\mathcal{P}) \geq c_{\mathcal{G}}$ für alle gültigen K -Zerlegungen gilt? Das wäre ein Analogon zum Isoperimetrischen Ungleichung auf Graphen (Cheeger-Ungleichung) und würde fundamentale Grenzen der erreichbaren Parallelisierungseffizienz bei PathSim-Modellen etablieren.

Erweiterung des Subgraph Algorithmus auf algebraische Schleifen. Die Voraussetzung der Azyklizität (Definition 7.2(c)) schließt Modelle mit algebraischen Schleifen aus. Algebraische Schleifen entstehen in PathSim, wenn ein Signalpfad ohne Gedächtniselement (ohne Integrator) einen Zykel im Graphen bildet; dies geschieht z.B. bei impliziten algebraischen Gleichungssystemen (differential-algebraische Gleichungen, DAE).

Für DAE-Systeme der Form $F(\dot{x}, x, u) = 0$ existieren Lösungsverfahren (BDF-Methoden, Radau-IIA [16]), die iterative Gleichungslöser verwenden. Der Subgraph Algorithmus müsste in diesem Fall:

- Zyklen im Graphen $\mathcal{G}$ identifizieren und als *strongly connected components* (SCCs) zusammenfassen.
- Jede SCC als einen einzigen Block behandeln, der intern durch einen iterativen Gleichungslöser ausgewertet wird.
- Die resultierende SCC-Kondensation als azyklischen Graphen behandeln und darauf den Subgraph Algorithmus anwenden.

Zu beweisen ist, dass diese Erweiterung die semantische Erhaltung (Theorem 7.12) auf DAE-Systeme überträgt; der Beweis erfordert, dass der iterative Gleichungslöser innerhalb jeder SCC korrekt konvergiert und die Toleranzen konsistent mit den Diskretisierungsfehlern aus Theorem 7.22 sind.

Subgraph Algorithmus für zeitkontinuierliche Compiler-Integration. Eine langfristige Vision ist die vollständige Integration des Subgraph Algorithmus in einen modernen C-Compiler als *linkerübergreifende Interprozedural-Analyse*. Hierbei würde der Compiler das Blockgraph-Modell nicht aus Python, sondern direkt aus annotierten C-Quellen extrahieren:

Listing 6: Annotiertes C-Format für den Subgraph Algorithmus

```

1  /* PathSim-Annotation: Blockgraph-Kanten fuer Subgraph Algorithmus */
2  __attribute__((pathsim_block(integrator, state="x1")))
3  static model_real_t integrate_x1(model_real_t xdot) {
4      return x1 + DT * xdot;
5  }
6
7  __attribute__((pathsim_block(gain, param=OMEGA_N)))
8  static model_real_t gain_omegan(model_real_t in) {
9      return OMEGA_N * in;
10 }
```

Der Compiler extrahiert aus diesen Annotierungen den Blockgraphen $\mathcal{G}$, wendet den Subgraph Algorithmus an und distribuiert die resultierenden Partitionen auf verfügbare Kerne – ohne dass der Entwickler explizit parallelisieren muss. Die Korrektheit dieses Ansatzes entspräche einem automatischen Korrektheitsbeweis auf Compilerebene: Der Compiler würde als *zertifiziertes Übersetzungswerkzeug* gelten, dessen Korrektheit direkt auf Theorem 7.27 fußt.

Diese Vision verbindet Compilerbau, Graphentheorie und formale Verifikation und bildet einen natürlichen Abschluss der in Kapitel 7 und Kapitel 6 entwickelten Theorie.

9.3.11 Erweitertes Typsystem: Vektorsignale und komplexe Zahlen

Das in Kapitel 5 entwickelte Typsystem beschränkt sich auf skalare `model_real_t`-Signale. Praktische Anwendungen erfordern:

- *Vektorsignale*: Busse und Vektorknoten ($u \in \mathbb{R}^k$) müssen als typisierte Arrays dargestellt werden. Das MISRA-Typsystem muss um Array-Typen erweitert werden; Rule 18.x (Pointer/Array) tritt in den Vordergrund. Für Vektorsignale ist Theorem 5.3 neu zu formulieren und zu beweisen.
- *Komplexwertige Signale*: Für Signalverarbeitungsanwendungen (FFT-Blöcke, I/Q-Demodulation) sind $z \in \mathbb{C}$ erforderlich. Da MISRA-C:2012 `_Complex`-Typen nicht direkt unterstützt, müssten komplexe Zahlen als Struct `{re, im}` mit expliziten Rechenregeln implementiert werden – ein vollständig neues Type-Safety-Theorem wäre notwendig.
- *Matrixsignale für Mehrkörpersimulation*: Rotationsmatrizen $R \in \text{SO}(3)$ und homogene Transformationsmatrizen $T \in \text{SE}(3)$ als Signaltypen erfordern ein Typsystem, das Gruppenstruktur (Orthogonalität, $\det = 1$) als statisch verifizierbare Invariante kodiert.

9.3.12 Lernbasierte Blockbibliotheken und KI-Integration

Eine zukunftsweisende Erweiterung betrifft datengetriebene Modellkomponenten:

- *Physics-Informed Neural Networks (PINNs)*: PINNs, die eine Differentialgleichung $\dot{x} = f(x, u)$ als Residuum im Loss kodieren, könnten als spezialisierte Integrator-Blöcke mit gelerntem Vektorfeld fungieren. Trainings- und Approximationsfehler müssen in die Fehleranalyse einbezogen werden.
- *Symbolische Regression zur Blockbibliotheks-Erweiterung*: Mittels PySR könnten aus Messdaten automatisch neue Blockklassen $\beta \in \mathcal{F}$ mit formaler Semantik identifiziert werden, die sich anschließend – ohne die Minimalität von $\mathcal{B}^*$ zu verletzen – als abgeleitete (nicht-primitive) Klassen in PathSim integrieren lassen.

9.3.13 Kompositionelle Korrektheit und Subsystem-Abstraktion

Industrielle PathSim-Modelle sind hierarchisch strukturiert. Zukünftige Arbeit sollte eine formale Theorie der *kompositionellen Korrektheit* entwickeln:

- *Subsystem-Blöcke*: Ein **Subsystem**-Block kapselt ein vollständiges PathSim-Modell $\mathcal{G}'$ als primitiven Block in $\mathcal{G}$. Korrektheit der Codegenerierung für $\mathcal{G}$ soll *kompositional* aus der Korrektheit der Teilgraphen folgen – formal basierend auf Assume-Guarantee-Reasoning.
- *Parametrische Korrektheit*: Eine parameterräumliche Korrektheit – “Für alle $\mu \in \mathcal{P}$ gilt $y_{\mathcal{G}(\mu)}(t) = y_{\Sigma(\mu)}(t)$ ” – würde Theorem 6.1 zu einer Familie von Modellen verallgemeinern und wäre für model-in-the-loop Variationsanalysen bedeutsam.
- *Inkrementelle Codegenerierung*: Bei kleinen Modifikationen an $\mathcal{G}$ (Parameteränderungen, Hinzufügen/Entfernen eines Blocks) sollte der Codegenerator nur den betroffenen Ausschnitt neu generieren. Ein Change-Impact-Analysealgorithmus auf dem Blockgraphen – mit formaler Korrektheitsdefinition – ist Gegenstand zukünftiger Untersuchungen.

Literatur

- [1] PathSim Development Team, *PathSim: A Python-based Block Diagram Simulation Framework*, <https://code.pathsim.org>, 2024.
- [2] MISRA Consortium, *MISRA-C:2012 – Guidelines for the Use of the C Language in Critical Systems*, 3rd ed., MIRA Ltd., 2012.
- [3] E. Hairer, S. P. Nørsett, G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed., Springer, 1993.
- [4] ISO, *ISO 26262: Road Vehicles – Functional Safety*, 2nd ed., ISO, 2018.
- [5] A. B. Kahn, “Topological sorting of large networks,” *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.
- [6] C. L. Liu, J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [7] Epp, S., *Subgraph Algorithmus*, 2026, Verfügbar bei <https://gitlab.com/epp-group/subgraph>.
- [8] C. T. Chen, *Linear System Theory and Design*, 3rd ed., Oxford University Press, New York, 1999.
- [9] P. J. Antsaklis, A. N. Michel, *A Linear Systems Primer*, Birkhäuser, Boston, 2007.
- [10] W. L. Brogan, *Modern Control System Theory and Design*, 2nd ed., Prentice Hall, Englewood Cliffs, NJ, 1991.
- [11] E. D. Sontag, *Mathematical Control Theory: Deterministic Finite-Dimensional Systems*, 2nd ed., Springer, New York, 1998.
- [12] H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, Upper Saddle River, NJ, 2002.
- [13] U. M. Ascher, L. R. Petzold, *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*, SIAM, Philadelphia, 1998.

- [14] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 2nd ed., Wiley, Chichester, 2008.
- [15] J. R. Dormand, P. J. Prince, “A family of embedded Runge-Kutta formulae,” *Journal of Computational and Applied Mathematics*, vol. 6, no. 1, pp. 19–26, 1980.
- [16] C. W. Gear, *Numerical Initial Value Problems in Ordinary Differential Equations*, Prentice Hall, Englewood Cliffs, NJ, 1971.
- [17] S. J. Mason, “Feedback theory – some properties of signal flow graphs,” *Proceedings of the IRE*, vol. 41, no. 9, pp. 1144–1156, 1953.
- [18] C. L. Coates, “Flow-graph solutions of linear algebraic equations,” *IRE Transactions on Circuit Theory*, vol. 6, no. 2, pp. 170–187, 1959.
- [19] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed., Springer, New York, 2011.
- [20] E. A. Lee, S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed., MIT Press, Cambridge, MA, 2017.
- [21] IEC, *IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems*, 2nd ed., IEC, Geneva, 2010.
- [22] L. Hatton, *Safer C: Developing Software for High-Integrity and Safety-Critical Systems*, McGraw-Hill, New York, 1995.
- [23] N. Storey, *Safety-Critical Computer Systems*, Addison-Wesley, Harlow, 1996.
- [24] E. M. Clarke, O. Grumberg, D. Peled, *Model Checking*, MIT Press, Cambridge, MA, 1999.
- [25] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, Cambridge, MA, 2009.
- [26] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, Boston, 2006.
- [27] ISO/IEC, *ISO/IEC 9899:1999 – Programming Languages – C*, ISO, Geneva, 1999.
- [28] IEEE, *IEEE 754-2019: IEEE Standard for Floating-Point Arithmetic*, IEEE, New York, 2019.
- [29] AUTOSAR Consortium, *AUTOSAR Classic Platform – Release R21-11*, <https://www.autosar.org>, 2021.
- [30] M. Barr, A. Massa, *Programming Embedded Systems: With C and GNU Development Tools*, 2nd ed., O’Reilly Media, Sebastopol, CA, 2006.
- [31] T. A. Henzinger, “The theory of hybrid automata,” *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pp. 278–292, 1996.
- [32] A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, Philadelphia, 2008.

- [33] D. S. Bernstein, *Matrix Mathematics: Theory, Facts, and Formulas*, 2nd ed., Princeton University Press, Princeton, NJ, 2009.